ManabiOps セットアップガイド

本番稼働までの完全手順書 (00-14)

Version 1.0

こども食堂「まなびキッチン」運営支援システム

発行日: 2026-04-22

リポジトリ: github.com/satoshiyoshimoto0426/manabi-kitchin

目次

00. 00. 概要・全体地図・事前準備

01. 01. LINE Messaging API セットアップ

02. 02. GCP プロジェクト・請求・サービスアカウント

03. 03. Gemini API キー取得

04. 04. Document AI プロセッサ作成 (レシート + 名簿)

05. 05. Firestore データベース作成

06. 06. Cloud Storage バケット作成

07. 07. Google Sheets 会計台帳作成

08. 08. Secret Manager にシークレット登録

09. 09. Cloud Run にアプリをデプロイ

10. 10. LINE Webhook URL を設定

11. 11. Cloud Scheduler (1時間バッチ) 設定

12. 12. Instagram Graph API 連携 (Phase 3)

13. 13. 動作確認・トラブルシューティング

14. 14. 運用開始チェックリスト

00. 概要・全体地図・事前準備

所要時間: 15分 (読むだけ)
このガイドは **「これから何をするのか」** を理解するためのものです。
ここを読まずに進めると迷子になりやすいので、必ず一度目を通してください。

☒このガイドで何をするか?

一言で言うと:

LINE公式アカウントに写真を送ると、AIが自動で会計・名簿・SNS投稿をしてくれるシステムを、インターネット上のGoogleのサーバーに設置して、本物のLINEと繋げる作業

☒全体地図 (完成後のイメージ)

```
┌──────────────────────────┐
│  スタッフのスマホ  │ ← 運営メンバーの LINE
└──────────────────────────┘
          │ 画像を送信
          ▼
┌──────────────────────────────┐
│  LINE 公式アカウント  │ ← ガイド 01 で作る
│  (まなびキッチン)     │
└──────────────────────────────┘
          │ Webhook (URL で通知)
          ▼
┌──────────────────────────────────────────────────┐
│  Cloud Run (Google のサーバー)  │ ← ガイド 09 でデプロイ
│  ManabiOps アプリが動く場所       │
└──────────────────────────────────────────────────┘
  │     │     │     │     │
  ▼     ▼     ▼     ▼     ▼
Gemini DocAI Firestore GCS  Sheets  Instagram
 (03)  (04)   (05)   (06) (07)    (12)
```

各ボックスが 1 つのセットアップガイドに対応しています。

⌧用語集

初見の単語を先に覚えておくと、以降のガイドがスムーズに読めます。

用語	ざっくり意味	例え話
GCP	Google Cloud Platform。Google が提供するサーバー貸しサービス	「サーバーのレンタル屋さん」
Cloud Run	GCPのサーバーの1つ。アプリを置いて動かせる	「アプリの自動販売機」
Firestore	GCPのデータベース。データを保存する場所	「デジタル金庫」
Cloud Storage	GCPのファイル置き場	「デジタル倉庫」
Document AI	GCPのOCR (画像から文字を読み取るAI)	「スキャナー+翻訳機」
Gemini	Google の AI。画像を見て説明できる	「賢い相談相手」
Secret Manager	パスワードやキーを安全に保管するGCPの金庫	「デジタル貸金庫」
Webhook	「何か起きたら特定のURLに通知してね」の仕組み	「出前の受付ベル」
API キー	サービスを使うための合言葉 (長い文字列)	「部屋の鍵」
プロジェクト	GCPでサービスをまとめて管理する単位	「1つの部屋」
サービスアカウント	人ではなくプログラムがGCPを使うためのアカウント	「ロボット用ID」
Looker Studio	無料のダッシュボード作成ツール	「無料のExcelグラフ」

⌧事前準備チェックリスト

作業を始める前に以下を準備してください。

必須

- [ ] **パソコン** (Windows / Mac どちらでもOK)
- [ ] **インターネット環境** (作業中は Zoom 等の重いアプリは閉じてください)
- [ ] **Gmail アカウント** 1つ
- 運営団体の共用アカウントを推奨 (個人アカウントだと退任時に面倒)
- まだ無い場合は https://accounts.google.com/signup で作成 (10分)
- [ ] **クレジットカード** 1枚
- GCP の請求用。**無料枠内なら引き落としは発生しません**が、登録は必須
- 個人カードでも可 (団体カードがベター)

- [ ] **スマホ** (LINE が入っているもの)
- LINE公式アカウントの管理・動作確認に使用
- [ ] **本ガイドを表示する端末** (パソコン以外、例: スマホ・タブレット)
- 手順を見ながら作業するため、画面がもう1つあると超便利

あるとベター

- [ ] **メモ帳アプリ** (パスワード保存用)
- 取得する API キーをコピーしておくため
- **セキュリティ重要**: 1Password, Bitwarden など**パスワードマネージャー**を推奨
- 最低限: PCのメモ帳に一時保存 → 作業完了後に削除
- [ ] **クラウドストレージ** (Google Drive など)
- サービスアカウントJSONファイルの保管用

☒セキュリティに関する重要な注意

本ガイドで取得する **API キー / サービスアカウント JSON** は、子どもたちの個人情報にアクセスできる**非常に重要な情報**です。以下を必ず守ってください。

やっていいこと

- ☒1Password などのパスワードマネージャーに保存
- ☒運営代表者と IT 担当者の 2名以内で共有
- ☒Secret Manager (ガイド 08) に登録して本番環境で使う

絶対にやってはいけないこと

- ☒**メール / LINE / SNS に貼り付けて送る** (絶対NG)
- ☒**GitHub などの公開リポジトリにコミット** (絶対NG)
- ☒**付箋に書いて PC に貼る** (情弱扱いされます)
- ☒**不要になったキーを放置** (必ず無効化する)

☒費用の考え方

初期費用

- **¥0** (クラウドサービスは使った分だけ課金)

月額費用 (想定: 月4回開催・画像50枚)

- **¥500〜¥3,000** (要件定義書の目標 ¥10,000 以内に収まる想定)

無料枠の範囲

各サービスには **無料枠** があります:
- Gemini API: 月 45万リクエスト程度は無料
- Document AI: 月1,000ページまで無料
- Cloud Run: 月200万リクエストまで無料
- Firestore: 月2万書込 × 30日 = 月60万書込まで無料

こども食堂の運用規模 (月4回 × 画像30枚程度) では、ほぼ無料枠内に収まります。

予算上限を設定する (推奨)

GCPで月額予算アラートを設定できます。ガイド02で実施します。
¥5,000 で警告、¥10,000 で機能停止 などに設定しておくと安心です。

⊠進め方の推奨

パターンA: 一気に1日で (集中型)

朝10時〜夕方16時 × 6時間確保して、Phase 1 まで一気に構築。
→ 推奨: 運営代表者 + IT 担当者で集まって作業

パターンB: 1日1ガイドずつ (じっくり型)

毎日夜に1時間ずつ、14日間で完成。
→ 推奨: 1人で作業・平日夜しか時間が取れない場合

パターンC: Phase分割 (慎重型)

まず Phase 1 (経理) だけ構築 → 1ヶ月運用 → Phase 2 (名簿) → 1ヶ月運用 → Phase 3 (SNS)
→ 推奨: リスクを最小化したい場合、要件定義書の推奨

🆘 詰まったら

1. エラー文を全文 Google で検索
2. 13_troubleshooting.md を確認

3. それでもダメなら **一旦作業を止めて** 開発担当者に連絡
   - 「ガイド ◯ の ステップ ◯ で、こういう画面が出て詰まりました」と連絡

無理して進めると設定がぐちゃぐちゃになる ので、迷ったら止まるのが鉄則です。

➡ 次のステップ

準備ができたら → **01. LINE Messaging API セットアップ**

一番最初は LINE の設定から始めます。ここは GCP より難易度が低く、無料で完結するので肩慣らしに最適です。

01. LINE Messaging API セットアップ

所要時間: **30分**
費用: **¥0** (月200通までの無料プラン)
このガイドで取得するもの: `LINE_CHANNEL_SECRET` と `LINE_CHANNEL_ACCESS_TOKEN`

☒このガイドでやること

1. LINE Business ID を作る
2. プロバイダー (運営団体の箱) を作る
3. Messaging API チャネル (公式アカウント本体) を作る
4. **チャネルシークレット** と **チャネルアクセストークン** を取得する
5. メモ帳に2つの値を記録する

ゴール

☒「まなびキッチン」という LINE 公式アカウントが作られて
☒プログラムから操作できる **2つの合言葉 (シークレット/トークン)** が手に入る

☒手順

Step 1. LINE Developers コンソールを開く

1. パソコンのブラウザで以下のURLを開く
   ☒**https://developers.line.biz/ja/**
2. 画面右上の **「ログイン」** をクリック
3. 「ビジネスアカウントでログイン」を選択
   - まだアカウントが無い場合 → **「LINE Business ID を作成」** をクリック
   - 画面の指示に従って作成 (メール or LINEアカウントで作成可)

Step 2. プロバイダーを作成する

プロバイダーとは: 公式アカウントを管理する「運営団体の箱」。1つ作れば複数のLINE公式アカウントを入れられます。

1. ログイン後、トップページに **「プロバイダー」** というセクションが表示される
2. **「作成」** ボタンをクリック (初回は「新規プロバイダー作成」)
3. プロバイダー名に **「まなびキッチン」** と入力
4. **「作成」** をクリック

> ☒プロバイダー名は後から変更できます。迷ったら団体名でOK。

Step 3. Messaging API チャネルを作成する

チャネルとは: LINE公式アカウントの本体。これが「友だち追加」できる対象になります。

1. プロバイダー画面に入ったら **「チャネルを作成」** の一覧から **「Messaging API」** を選択
2. 以下の項目を入力:

項目	入力する値
チャネルの種類	Messaging API (選択済のはず)
プロバイダー	まなびキッチン (選択済のはず)
会社・事業者の所在国・地域	日本
チャネルアイコン	まなびキッチンのロゴ画像をアップロード (後で変更可)
チャネル名	**まなびキッチン**
チャネル説明	「こども食堂の運営支援システムです」など
大業種	「非営利団体」 or 「教育」
小業種	「NPO・公益法人」など近いもの
メールアドレス	運営団体の連絡用メールアドレス
プライバシーポリシーURL	空欄でOK (後で追加可)
サービス利用規約URL	空欄でOK

1. 利用規約に同意するチェックボックスをすべて ☒
2. **「作成」** をクリック

> ☒この時点で公開前のLINE公式アカウントが作られました。まだ友だち追加はできません。

Step 4. チャネル基本情報から チャネルシークレット を取得

1. 作成されたチャネルの画面が開く
2. 上部のタブから **「チャネル基本設定」** を選択
3. 下にスクロールして **「チャネルシークレット」** の項目を探す
4. 表示されている文字列 (例: `a1b2c3d4e5f6g7h8...`) を **コピー** する
5. メモ帳に以下のように貼り付け:

`=== ManabiOps API キー === LINE_CHANNEL_SECRET = (ここに貼り付け)`

> ☒**チャネルシークレットは絶対に他人に見せない** こと。

Step 5. チャネルアクセストークン を発行

1. 同じチャネル画面の上部タブ **「Messaging API設定」** を選択
2. 画面を一番下までスクロール
3. **「チャネルアクセストークン (長期)」** のセクションに **「発行」** ボタンがある → クリック
4. 発行された長い文字列 (200文字くらい) が表示される → **コピー**
5. メモ帳に追記:

`=== ManabiOps API キー === LINE_CHANNEL_SECRET = ... LINE_CHANNEL_ACCESS_TOKEN = (ここに貼り付け)`

> ☒このトークンは**1回しか表示されない**ので、必ずコピーしてください。
> 万一失くしても「再発行」できますが、既存の設定は全て無効になります。

Step 6. 応答設定を確認する

ここが重要: デフォルトでは「自動応答メッセージ」が有効になっており、ManabiOps の動作を妨害します。**必ずOFFにしてください**。

1. **「Messaging API設定」** タブの中段にある **「応答設定」** を見つける
2. **「応答メッセージ」** → **無効** に変更
3. **「あいさつメッセージ」** → 有効のままでOK (初回友だち追加時の挨拶)

4. **「Webhook」** → **有効** にする
   - URL はまだ空欄でOK (ガイド 10 で設定します)

完了時の応答設定

項目	設定値
応答メッセージ	☒無効
あいさつメッセージ	☒有効 (任意)
Webhook	☒有効

Step 7. あいさつメッセージを設定 (任意)

1. 「あいさつメッセージ」の右側の **「編集」** をクリック
2. 新規メッセージを作成。例:
   ```
   ☒まなびキッチンの運営サポートBotへようこそ!

   レシート・名簿・イベント写真をこのトークに送信すると、
   AIが自動で整理してお知らせします。

   まずは運営代表者に、利用者登録を依頼してください。
   ```
3. 保存

Step 8. QRコードと ベーシックID の取得

1. **「Messaging API設定」** タブの上部に以下の情報がある:
   - **Bot basic ID**: `@xxxxxxxxx` (友だち追加用ID)
   - **QRコード**: 画像をダウンロード

メモ帳に追記:

```
=== ManabiOps LINE 公式アカウント情報 ===
Bot basic ID: @xxxxxxxxx
QRコード: (画像ファイルを団体共有フォルダに保存)
```

これらはスタッフに「友だち追加してね」と案内する時に使います。

☒このガイド完了時のチェックリスト

- [ ] LINE Business ID でログインできた
- [ ] プロバイダー「まなびキッチン」を作成した
- [ ] Messaging API チャネル「まなびキッチン」を作成した
- [ ] **LINE_CHANNEL_SECRET** をメモ帳に記録した
- [ ] **LINE_CHANNEL_ACCESS_TOKEN** をメモ帳に記録した
- [ ] 応答メッセージ: **無効**
- [ ] Webhook: **有効** (URL は空欄でOK)
- [ ] Bot basic ID と QRコードを保存した

🆘 よくあるトラブル

Q1. 「ビジネスアカウントが必要です」と表示される

→ 上記 Step 1 で LINE Business ID を作成してください。個人の LINE アカウントとは別物です。

Q2. 「チャネルアクセストークン」の「発行」ボタンが見つからない

→ タブが **「Messaging API設定」** になっているか確認 (「チャネル基本設定」と間違えやすい)。
→ 画面を一番下までスクロールしてください。

Q3. シークレットやトークンを失くしてしまった

- **シークレット**: 「チャネル基本設定」タブで確認可能 (失くしても表示は消えない)
- **トークン**: 「再発行」ボタンで新しいものを発行可能。**ただし古いトークンは無効になる**ので、後続ガイドで設定する前なら問題なし。

Q4. 「料金が発生しそうで怖い」

→ フリープラン (月200通) で今は十分です。月4回開催×メッセージ50通 = 月200通程度。
→ 利用通数は LINE Official Account Manager で確認できます。

➡ 次のステップ

LINE の準備ができたら → **02. GCP プロジェクト作成**

次は Google Cloud 側の準備です。クレジットカードの登録が必要になりますが、**無料枠内なら引き落としは発生しません**。

02. GCP プロジェクト・請求・サービスアカウント

所要時間: **40分**
費用: **¥0** (無料枠内の使用想定、初回$300クレジット付与あり)
このガイドで取得するもの: `GCP_PROJECT_ID` , サービスアカウントJSONファイル

☒このガイドでやること

1. GCP (Google Cloud Platform) にアクセス
2. 請求先アカウント (クレジットカード登録) を作成
3. 「manabi-ops」プロジェクトを作成
4. 必要な API を一括で有効化
5. サービスアカウントを作成して JSON キーをダウンロード
6. **月額予算アラート**を設定 (超重要)

ゴール

☒GCP のプロジェクト `manabi-ops` が作られている
☒プログラムが GCP を操作するための「ロボット用ID」(サービスアカウント) が用意されている
☒予期せぬ高額請求を防ぐアラートが設定されている

☒手順

Step 1. GCP にアクセス

1. ブラウザで以下を開く
   ☒**https://console.cloud.google.com/**
2. Gmail アカウントでログイン (ガイド00で準備したもの)
3. 初回は「利用規約に同意」のチェックボックスに ☒→ 「同意して続行」

Step 2. 請求先アカウントを作成 (初回のみ)

> ⌧**重要**: クレジットカードが必要ですが、**初回は $300 (約4.5万円) の無料クレジットが90日間自動付与**されます。この範囲なら一切課金されません。

1. 画面左上のメニュー (☰) → **「お支払い」** をクリック
2. **「請求先アカウントを追加」** または **「請求先アカウントを作成」** をクリック
3. 情報を入力:

項目	入力する値
アカウント種別	個人 or ビジネス (団体の場合はビジネス)
名前	代表者氏名 or 団体名
住所	団体住所
電話番号	連絡可能な番号
支払い方法	クレジットカード情報

1. **「送信して課金を有効にする」**

> ⌧$300クレジットが付与されると「Free Trial credit: $300.00」という表示が出ます。

Step 3. プロジェクトを作成

プロジェクトとは: GCP でサービスをまとめる「部屋」のようなもの。

1. 画面上部の青いバーの左側にある **「プロジェクトを選択」** のプルダウンをクリック
2. 表示されるダイアログの右上の **「新しいプロジェクト」** をクリック
3. 以下を入力:

項目	入力する値
プロジェクト名	`manabi-ops`
プロジェクト ID	自動生成 (例: `manabi-ops-432501`) → **これをメモ帳に記録**
組織	選択不要 (なしでOK)
場所	選択不要

1. **「作成」** をクリック → 30秒ほど待つ

2. 通知アイコンに「プロジェクトが作成されました」と表示されたら、画面上部のプロジェクト名を **「manabi-ops」** に切り替える

メモ帳に追記:

```
=== GCP ===
GCP_PROJECT_ID = manabi-ops-432501 （ここは皆さんのIDに置き換え）
```

Step 4. 請求先アカウントをプロジェクトに紐づけ

1. 画面左メニュー (☰) → **「お支払い」** をクリック
2. **「請求先アカウントをリンク」** をクリック
3. Step 2 で作った請求先アカウントを選択して **「アカウントを設定」**

> ⚠ これをやらないと Cloud Run デプロイ時にエラーになります。

Step 5. 月額予算アラートを設定 (超重要 ⛔)

> ⛔**必ずやってください**。これをやらないと、万一バグで無限ループが発生した時に高額請求が来ます。

1. 左メニュー → **「お支払い」** → **「予算とアラート」**
2. **「予算を作成」** をクリック
3. 入力:

項目	入力する値
名前	`manabi-ops月額予算`
予算額	**10000 円** (月額)
対象期間	月次
プロジェクト	manabi-ops

1. **「次へ」**
2. 通知しきい値を以下に設定:
   - **50%** で通知 (¥5,000到達時)
   - **90%** で通知 (¥9,000到達時)
   - **100%** で通知 (¥10,000到達時)
3. 通知メールに **Gmail アドレス** を入力

4. **「完了」**

これで予算の 50%, 90%, 100% に達した時点で Gmail にアラートが届きます。

Step 6. 必要な API を有効化

ManabiOps が使う Google のサービスを一括で「使えるようにする」作業です。

1. 左メニュー → **「API とサービス」** → **「ライブラリ」**
2. 以下を**1つずつ**検索して、それぞれ **「有効にする」** をクリック:

#	API 名	検索ワード	用途
1	**Cloud Run Admin API**	`cloud run admin`	アプリの実行 (ガイド09)
2	**Cloud Build API**	`cloud build`	デプロイ時のビルド
3	**Artifact Registry API**	`artifact registry`	Dockerイメージ保管
4	**Firestore API**	`firestore api`	データベース (ガイド05)
5	**Document AI API**	`document ai`	OCR (ガイド04)
6	**Generative Language API**	`generative language`	Gemini AI (ガイド03)
7	**Cloud Storage API**	`cloud storage api`	ファイル保管 (ガイド06)
8	**Google Sheets API**	`google sheets api`	会計台帳 (ガイド07)
9	**Secret Manager API**	`secret manager`	パスワード保管 (ガイド08)
10	**Cloud Scheduler API**	`cloud scheduler`	バッチ実行 (ガイド11)
11	**Cloud Logging API**	`cloud logging`	ログ確認

> ☒各 API は有効化に 30秒〜1分 かかります。有効化完了を待ってから次のAPIに進んでください。

一括コマンド版 (上級者向け)

Cloud Shell が使える方は、画面右上の ">_" マーク (Cloud Shell) を開いて以下を貼り付けると一発で有効化できます:

```
gcloud services enable \
  run.googleapis.com \
  cloudbuild.googleapis.com \
  artifactregistry.googleapis.com \
  firestore.googleapis.com \
  documentai.googleapis.com \
  generativelanguage.googleapis.com \
  storage.googleapis.com \
  sheets.googleapis.com \
  secretmanager.googleapis.com \
  cloudscheduler.googleapis.com \
  logging.googleapis.com
```

Step 7. サービスアカウントを作成

サービスアカウントとは: プログラム (Cloud Run) が GCP を操作するための「ロボット用ID」。

1. 左メニュー → **「IAM と管理」** → **「サービスアカウント」**
2. 上部の **「+ サービスアカウントを作成」** をクリック
3. 入力:

項目	入力する値
サービスアカウント名	`manabi-ops-runner`
サービスアカウントID	自動 (manabi-ops-runner)
説明	`ManabiOps の Cloud Run 実行用`

1. **「作成して続行」**
2. **「ロールを選択」** で以下を**全て追加** (1つずつ選択 → 「別のロールを追加」を繰り返し):

ロール名	用途
Cloud Datastore ユーザー	Firestore 読み書き
Storage オブジェクト管理者	Cloud Storage 読み書き
Document AI API ユーザー	DocumentAI 呼び出し
Secret Manager のシークレット アクセサー	Secret 取得
ログ書き込み	Cloud Logging

1. **「続行」** → **「完了」**

Step 8. サービスアカウントの JSON キーをダウンロード

> ⚠ このJSONファイルは**超重要**。厳重管理してください。

1. 作成されたサービスアカウント `manabi-ops-runner@...` をクリック
2. 上部タブ **「キー」** をクリック
3. **「鍵を追加」** → **「新しい鍵を作成」**
4. **キーのタイプ: JSON** を選択 → **「作成」**
5. JSON ファイルが自動ダウンロードされる (例: `manabi-ops-432501-abc123.json`)

JSONファイルの扱い

- [ ] ファイル名を `manabi-ops-sa-key.json` にリネーム
- [ ] 運営団体の Google Drive の**共有されていない**フォルダに保存
- [ ] パスワードマネージャーにバックアップを保存 (1Password の「セキュアノート」に添付など)
- [ ] **絶対に Slack / LINE / Gmail 等に貼り付けて送らない**
- [ ] **絶対に GitHub にコミットしない**

メモ帳に追記:

```
=== GCP サービスアカウント ===
メールアドレス: manabi-ops-runner@manabi-ops-432501.iam.gserviceaccount.com
JSON キー保存場所: (Google Drive の URL やパス)
```

Step 9. 動作確認 (Cloud Shell で)

画面右上のヘッダーに **「>_」(Cloud Shell)** アイコンがある → クリックして起動

```
# プロジェクトを確認
gcloud config list project

# 有効な API 一覧
gcloud services list --enabled | grep -E "(run|firestore|documentai|sheets|storage|secretmanager)"

# サービスアカウント一覧
gcloud iam service-accounts list
```

manabi-ops-runner が表示されれば OK。

☒このガイド完了時のチェックリスト

- [ ] GCP にログインできた
- [ ] 請求先アカウントを作成・紐付け済み
- [ ] プロジェクト `manabi-ops` 作成済み → **プロジェクト ID メモ済み**
- [ ] 月額予算アラート設定済み (¥10,000 / 50・90・100%)
- [ ] 11個の API を全て有効化済み
- [ ] サービスアカウント `manabi-ops-runner` 作成済み
- [ ] 5つのロールを付与済み
- [ ] サービスアカウント JSON キーをダウンロードして**安全な場所に保存**済み

🆘 よくあるトラブル

Q1. クレジットカード登録でエラーが出る

- デビットカードやプリペイドカードは使えません (クレジットカードのみ)
- カード会社の不正利用検知に引っかかっている可能性 → カード会社に連絡

Q2. プロジェクト作成で「組織が必要」と出る

- 個人アカウントなら「組織」を選択せず、「組織なし」のままでOK

Q3. 「$300 無料クレジット」がどこにあるかわからない

- 左メニュー → お支払い → 概要 の右側に **「無料トライアル」** というセクションがあります

Q4. API有効化で「権限がありません」と出る

- プロジェクトを切り替えているか確認 (画面上部の青いバーの左に「manabi-ops」と出ているか)

Q5. サービスアカウント JSON をうっかり GitHub に公開してしまった

1. 即座に GCP Console でそのキーを**削除** (IAM → サービスアカウント → 鍵)
2. 新しい鍵を作成
3. 既に使われていないか Cloud Logging を確認
4. 詳しくは開発担当者に相談

➡ 次のステップ

GCP 基盤ができたら → **03. Gemini API キー取得**

次は 10 分で終わる、Gemini (AI) の鍵を取得する簡単なガイドです。

03. Gemini API キー取得

所要時間: **10分**
費用: **¥0** (月 45万リクエスト程度まで無料)
このガイドで取得するもの: `GEMINI_API_KEY`

このガイドでやること

1. Google AI Studio にアクセス
2. API キーを作成
3. メモ帳に記録

ゴール

AI (Gemini) を呼び出すための合言葉が手に入る

手順

Step 1. Google AI Studio にアクセス

1. ブラウザで以下を開く
   https://aistudio.google.com/app/apikey
2. **ガイド02 と同じ Gmail アカウント** でログイン
3. 利用規約に同意 (初回のみ)

Step 2. API キーを作成

1. 画面左上または中央に **「Create API key」** (または「APIキーを作成」) ボタン → クリック
2. ポップアップで **「Create API key in existing project」** を選択
3. プロジェクトのプルダウンで `manabi-ops` (ガイド02で作成) を選択
4. **「Create API key」** をクリック
5. 作成されたキー (例: `AIzaSyD...` で始まる 40文字くらい) が表示される → **コピー** する

メモ帳に追記:

```
=== Gemini API ===
GEMINI_API_KEY = AIzaSyD... (ここに貼り付け)
```

Step 3. 使用するモデルを確認

ManabiOps は **Gemini 2.5 Flash** を使用します (要件定義書第4章 FR-01)。

- 画面左メニューの **「Models」** または **「モデル」** でモデル一覧を確認できます
- `gemini-2.5-flash` が利用可能であることを確認
- このモデルは高速・低コスト・マルチモーダル対応 (画像理解OK)

Step 4. 動作確認 (任意・推奨)

1. AI Studio のトップで **「Create new prompt」** または **「Chat」** をクリック
2. 右上のモデルを `gemini-2.5-flash` に設定
3. 日本語でテストメッセージを入力:
   `こんにちは。レシートの画像から勘定科目を推論するタスクを任されています。`
4. AIが返答すれば API は正常に動作しています

☒このガイド完了時のチェックリスト

- [ ] Google AI Studio にログインできた
- [ ] `manabi-ops` プロジェクトに紐づいた API キーを作成した
- [ ] `GEMINI_API_KEY` をメモ帳に記録した
- [ ] gemini-2.5-flash モデルが利用可能であることを確認した

☒豆知識

Gemini API の無料枠

- **無料枠**: 15 RPM (1分あたり15リクエスト) / 100万トークン/日 / 1,500リクエスト/日
- **こども食堂の規模**: 月4回 × 画像30枚程度なら完全に無料枠内

API キーの管理ベストプラクティス

- **本番用キー**と**開発用キー**は分けるのが理想 (今回は1つでOK)
- 万一漏洩したら → AI Studio の API key 画面で「Delete」してすぐ作り直し

料金監視

- ガイド02で設定した月額予算アラート (¥10,000) がここでも効きます
- 想定外の使用があっても自動で通知が来るので安心

🆘 よくあるトラブル

Q1. 「Create API key」ボタンが見つからない

- ログインしている Google アカウントを確認
- 一旦ログアウトして、ガイド02と同じアカウントでログインし直す

Q2. プロジェクト選択で `manabi-ops` が出てこない

- ガイド02のStep3 (プロジェクト作成) が完了しているか確認
- ブラウザをリロード (5分ほど反映に時間がかかる場合あり)

Q3. 「API key creation failed」エラー

- ガイド02のStep6で **「Generative Language API」** を有効化したか確認
- していなければ GCP Console で有効化してから再試行

Q4. APIキーが "Expired" になった

- 無料枠の API キーは期限なし
- もし期限付きで作成してしまった場合は削除して作り直し

➡ 次のステップ

Gemini の準備ができたら → **04. Document AI プロセッサ作成**

次はレシートと名簿の OCR プロセッサを作成します。**Phase 1 MVP のキモ**です。

04. Document AI プロセッサ作成 (レシート + 名簿)

所要時間: **60分** (プロセッサ作成) + **PoC 2週間** (名簿プロセッサの訓練・要件定義書 Phase 0)
費用: レシート $0.10/ページ、名簿(Custom) $0.30/ページ (月1,000ページまで無料)
このガイドで取得するもの: `DOCAI_RECEIPT_PROCESSOR_ID` , `DOCAI_ROSTER_PROCESSOR_ID`

☒このガイドでやること

1. **Expense Parser** (レシート専用) プロセッサを作成 → すぐ使える
2. **Custom Extractor** (名簿用) プロセッサを作成 → **実データで訓練が必要** (Phase 2まで延期可)

ゴール

☒レシートOCRがすぐ動く (Phase 1 MVP)
☒名簿OCRの訓練用ひな形が用意できる (Phase 2)

☒重要な前提: 2つのプロセッサ

プロセッサ	用途	難易度	精度	Phase
Expense Parser	レシート・領収書	☒超簡単 (訓練不要)	90%以上	Phase 1
Custom Extractor	手書き名簿	☒☒☒訓練が必要	訓練次第	Phase 2

本ガイドでは両方を扱いますが、**Custom Extractor は手書き名簿サンプルが20〜50枚必要**なので、最初はレシートだけ作ってPhase 1を稼働させる方針をおすすめします。

パート1: レシートプロセッサ作成 (Phase 1 必須)

Step 1. Document AI コンソールを開く

1. GCP Console で、画面左メニュー (☰) → **「Document AI」**
   - 左メニューで見つからない場合は、上部検索バーで `Document AI` と検索
2. プロジェクトが `manabi-ops` になっていることを確認
3. 「Document AI の使用を開始する」画面が出たら **「有効にする」** をクリック (初回のみ)

Step 2. Expense Parser プロセッサを作成

1. 左メニューの **「プロセッサ ギャラリー」** (Processor Gallery) をクリック
2. 検索ボックスに `expense` と入力
3. **「Expense Parser」** というカードが出る → クリック
4. 右側パネルで **「プロセッサを作成」** (Create Processor) をクリック
5. 入力:

項目	入力する値
プロセッサ名	`manabi-receipt-parser`
リージョン	**US (us)** を選択 ※ 現時点でExpense Parserは US のみ

1. **「作成」** をクリック
2. 作成完了したプロセッサの画面に遷移する

Step 3. プロセッサ ID をコピー

1. プロセッサ詳細画面の上部に以下が表示される:
   - **プロセッサ ID**: 20桁くらいの英数字 (例: `abc123def456ghi789jk`)
   - **プロセッサ エンドポイント**: `us-documentai.googleapis.com` 等

2. プロセッサIDを **コピー**

メモ帳に追記:

```
=== Document AI ===
DOCAI_LOCATION = us
DOCAI_RECEIPT_PROCESSOR_ID = abc123def456ghi789jk (ここに貼り付け)
```

Step 4. レシートプロセッサの動作テスト (推奨)

1. プロセッサ詳細画面の **「テストとサンプル」** タブ (または「Test & Samples」) をクリック
2. **「ドキュメントをアップロード」** → 手持ちのレシート写真 (JPEG/PNG/PDF) をアップロード
3. 数秒後、画面右側に抽出結果が表示される:
   - 店舗名 (supplier_name)
   - 合計金額 (total_amount)
   - 日付 (receipt_date)
   - 品目リスト (line_items)

☒これで OK。精度が低い (70%未満) 場合は、より明るい場所で撮影したレシートで試してください。

パート2: 名簿プロセッサ作成 (Phase 2 で必要)

> ⊠**推奨**: Phase 1 リリース優先なら、このパート2は**後回し**にしてOK。
> まず `DOCAI_ROSTER_PROCESSOR_ID` を空欄のままデプロイすると、名簿OCRは自動でMOCK動作します。

Step 5. Custom Extractor プロセッサを作成

1. プロセッサギャラリー → `custom extractor` で検索
2. **「Custom Extractor」** カード → **「プロセッサを作成」**
3. 入力:

項目	入力する値
プロセッサ名	`manabi-roster-extractor`
リージョン	**us** (推奨) or **asia-northeast1**

1. **「作成」**

Step 6. 抽出スキーマを定義

プロセッサ画面で **「管理」** タブ → **「スキーマ」** → **「スキーマを編集」**

以下のフィールドを追加:

フィールド名	データ型	出現回数	説明
`participant`	種類: `ネストされたエンティティ`	複数 (many)	名簿の1行分
`participant/name`	文字列	1回	氏名
`participant/category`	文字列	1回	区分 (大人/こども)
`participant/fee`	数値	0-1回	利用料金

手順詳細:
1. 「フィールドを追加」 → 名前を `participant`、種類を「ネストされたエンティティ」、出現回数を「複数」
2. `participant` の中に「子フィールドを追加」 で `name`, `category`, `fee` を追加

> ⚠ この**スキーマ名は必ず上記の通り**（`participant`, `name`, `category`, `fee`）にしてください。コード (src/services/documentai.ts) でこの名前で参照しています。

Step 7. 訓練データのアップロード (PoC 2週間 相当)

> ☒ここからが要件定義書 Phase 0 (PoC) に該当する作業です。

7.1 サンプル収集

- 過去に使った手書き名簿を **最低 20枚**、理想 **50枚** 収集
- iPhone/Android で撮影。明るい場所・文字が読める程度の解像度で
- JPEG/PNG 形式、1ファイル 10MB 以下

7.2 アップロード

1. プロセッサ画面 → **「訓練」** タブ → **「ドキュメントをアップロード」**
2. 20〜50枚を一括アップロード
3. 各ドキュメントが **「未ラベル」** 状態になる

7.3 ラベル付け (手動アノテーション)

1. 各ドキュメントをクリック → エディタ画面が開く
2. 画像上の氏名テキストをドラッグ選択 → **「name」** タグを選択
3. 区分部分をドラッグ選択 → **「category」** タグを選択
4. 利用料金部分をドラッグ選択 → **「fee」** タグを選択
5. **「保存」**

> ☒**1枚あたり 3〜5分**、20枚で**1〜2時間**、50枚で**3〜4時間**。運営メンバーで分担推奨。

7.4 データセット分割

20枚以上ラベル付けしたら:
1. **「データセット」** タブ → 自動で訓練用・テスト用に分割される
2. **訓練用: 80%、テスト用: 20%** が理想

7.5 モデル訓練

1. **「訓練」** タブ → **「新しいバージョンを訓練」**
2. バージョン名: `v1-initial`
3. **「訓練」** をクリック → **1〜3時間待機**
4. 完了後、精度スコア (F1) が表示される

7.6 精度評価と再訓練

- **F1 スコア 0.85 以上** が目標
- 0.85 未満の場合:
- 誤判定されているケースを確認して追加訓練データをアップロード
- ラベル付けミスを修正
- 再訓練

7.7 デプロイ

1. 精度が十分になったら **「デプロイ」** ボタンをクリック
2. 数分後、デプロイ完了

Step 8. 名簿プロセッサ ID をコピー

1. プロセッサ画面の上部に **プロセッサ ID** が表示される
2. コピーしてメモ帳に追記:

```
=== Document AI ===
DOCAI_LOCATION = us
DOCAI_RECEIPT_PROCESSOR_ID = abc123def456ghi789jk
DOCAI_ROSTER_PROCESSOR_ID = xyz789abc123def456gh
```

☒このガイド完了時のチェックリスト

Phase 1 (レシートのみ)

- [ ] Expense Parser プロセッサ `manabi-receipt-parser` 作成済み
- [ ] `DOCAI_RECEIPT_PROCESSOR_ID` メモ済み
- [ ] 手持ちのレシート写真で動作テスト済み (精度90%以上)

Phase 2 (名簿を追加する場合)

- [ ] Custom Extractor プロセッサ `manabi-roster-extractor` 作成済み

- [ ] スキーマ定義済み (participant/name/category/fee)
- [ ] 訓練データ 20〜50枚アップロード・ラベル付け済み
- [ ] F1 スコア 0.85 以上
- [ ] プロセッサをデプロイ済み
- [ ] `DOCAI_ROSTER_PROCESSOR_ID` メモ済み

🆘 よくあるトラブル

Q1. Expense Parser が「リージョンが選択できません」

- 現時点ではExpense Parserは **us** リージョンのみ対応
- asia-northeast1 は使えないため us を選択

Q2. レシートの精度が低い

- 撮影角度を正面に
- 影が入らないよう明るい場所で
- レシートを平らに置いて撮影
- スマホのカメラでピントを合わせる

Q3. 名簿の Custom Extractor の訓練に時間がかかりすぎる

- **Phase 1 段階では名簿プロセッサを作らない**選択でOK
- その場合は `.env` で `DOCAI_ROSTER_PROCESSOR_ID` を空欄のままにする
- 名簿OCRはMOCK動作 (ダミーデータを返す) になります
- Phase 2 で改めて訓練

Q4. 訓練で「ドキュメント数が不足」エラー

- 最低 10枚が必要 (推奨 20〜50枚)
- 訓練用と評価用に分けるので、実質 25枚以上推奨

Q5. プロセッサ ID がどこにあるかわからない

- プロセッサ詳細画面の**上部バー**に表示される
- URL の末尾の 20桁の文字列でも可 (`/processors/xxx` の xxx 部分)

☒豆知識: プロセッサの種類

プロセッサ	費用	学習不要	用途
Expense Parser	$0.10/page	☒	レシート・領収書
Invoice Parser	$0.10/page	☒	請求書
Form Parser	$0.05/page	☒	フォームの汎用抽出
OCR Processor	$0.015/page	☒	単純な文字抽出
Custom Extractor	$0.30/page	☒要学習	独自フォーマット用

名簿の代替案として、最初は **Form Parser** を使い、ある程度データが貯まったら Custom Extractor に切り替える、という戦略も有効です。

➡ 次のステップ

Document AI の準備ができたら → **05. Firestore データベース作成**

次は 15 分でデータベースを作成します。ボタンを数回押すだけの簡単ステップです。

05. Firestore データベース作成

所要時間: **15分**
費用: 無料枠 (1日2万書込まで無料) — まなびキッチン規模では **ほぼ¥0**
このガイドで取得するもの: Firestore が使える状態 (取得する値は無し)

☒このガイドでやること

Firestore は ManabiOps が **参加者名簿・取引履歴・承認待ちデータ** を保存するデータベースです。
ここではデータベースを作成し、セキュリティルールを設定します。

ゴール

☒ `manabi-ops` プロジェクトに Firestore (ネイティブモード) が作成される
☒リージョンは `asia-northeast1 (東京)`
☒初期セキュリティルールが設定される

☒重要な用語

用語	意味
コレクション	データの「フォルダ」のようなもの (例: `participants` = 参加者一覧)
ドキュメント	1件のレコード (例: 田中太郎さんの情報)
ネイティブモード	今から作る標準モード (推奨)
Datastore モード	旧方式 — **使いません**

Step 1. Firestore コンソールを開く

1. GCP Console を開き、画面上部でプロジェクトが `manabi-ops` になっていることを確認
2. 画面左メニュー (☰) → **「Firestore」**
   - 左メニューに無い場合は、上部検索バーで `Firestore` と検索

Step 2. データベースを作成

初回は「データベースを作成」画面が表示されます。

1. **「データベースを作成」** (Create database) をクリック
2. **モード選択** の画面で:
   - **「ネイティブモード」** を選択 (☒必ずこちら)
   - ☒「Datastore モード」は選ばないでください
3. **「続行」** をクリック
4. **ロケーション** 選択で:
   - 種類: **「リージョン」** (Region)
   - リージョン: `asia-northeast1 (Tokyo)` ☒東京を選択
   - ⚠ **ロケーションは後から変更できません** — 必ず東京を選択
5. **「データベースを作成」** をクリック
6. 1〜2分待つとデータベースが作成される

Step 3. コレクションは自動作成される

ManabiOps のコードが動き出すと、以下のコレクションが自動的に作られます。
この画面で手動作成する必要はありません。

コレクション名	用途	主なフィールド
`events`	イベント(こども食堂開催日)	`eventId` , `date` , `adultCount` , `childCount` , `totalRevenue`
`participants`	参加者名簿 (氏名は暗号化)	`participantId` , `nameEncrypted` , `nameHash` , `category`
`transactions`	収支記録	`txId` , `date` , `type` , `amount` , `vendor` , `receiptUrl`
`media`	受信した画像/動画	`mediaId` , `type` , `gcsPath` , `uploadedBy`
`posts`	Instagram投稿	`postId` , `caption` , `mediaIds` , `approvedBy`
`users`	LINE ホワイトリスト	`lineUserId` , `role` , `displayName`
`pendingApprovals`	承認待ちデータ	`approvalId` , `type` , `payload` , `status`

Step 4. セキュリティルールを設定

重要: 初期状態では「30日後にすべて拒否」の仮ルールになっています。本番運用に合わせて変更します。

1. 左メニュー → **「ルール」** (Rules) タブ
2. 以下のルールをコピーして貼り付け:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    // ManabiOps のサーバーサイド (Cloud Run) からのみアクセス
    // サービスアカウント経由のアクセスは Admin SDK が使用するため
    // このルールに関係なく通る。クライアントからの直接アクセスは全て拒否。
    match /{document=**} {
      allow read, write: if false;
    }
  }
}
```

1. **「公開」** (Publish) をクリック

> ☒**なぜ全拒否でよいのか**:
> ManabiOps のサーバーは **Firebase Admin SDK** を使用します。Admin SDK はこのセキュリティルールを **バイパス** してアクセスできるため、サーバーは問題なく動作します。
> 一方、悪意のある第三者がブラウザから直接 Firestore にアクセスしようとしても、このルールで **100% 拒否** されます。

Step 5. 動作確認

1. 左メニュー → **「データ」** (Data) タブに戻る
2. 画面上部に **「asia-northeast1」** と表示されていればOK
3. 空のコレクション一覧が表示されれば準備完了

☒完了チェック

- [ ] Firestore データベースが **ネイティブモード** で作成された
- [ ] ロケーションは **asia-northeast1** である
- [ ] セキュリティルールが「全拒否」で公開された
- [ ] エラー画面が表示されていない

☒次のステップ

→ **06_cloud_storage.md** — Cloud Storage バケット作成へ進む

🆘 トラブルシューティング

☒「データベースを作成」ボタンがグレーアウトして押せない

- 請求アカウントがリンクされていない可能性 → 02_gcp_project.md Step 3 を再確認
- API が未有効 → 上部検索バーで `Firestore API` → **有効にする**

☒「このロケーションには Firestore を作成できません」

- `asia-northeast1` を選んでいるか確認
- プロジェクト ID にスペースや日本語が含まれていないか確認

☒後からロケーションを変更したい

- **Firestore のロケーションは変更できません**
- 対処法: プロジェクトごと削除して `02` からやり直し (データが無い状態でのみ推奨)

06. Cloud Storage バケット作成

所要時間: **15分**
費用: **5GB まで無料** — まなびキッチン規模 (月10GB想定) で **月¥15〜¥30**
このガイドで取得するもの: `GCS_BUCKET_NAME` = **`manabi-ops-media`**

☒このガイドでやること

Cloud Storage は ManabiOps が **LINE で受信した画像・動画・OCR前のレシート原本** を保存するストレージです。
ここでは専用バケット (=フォルダのようなもの) を1つ作成します。

ゴール

☒ `manabi-ops-media` という名前のバケットが作成される
☒リージョンは `asia-northeast1 (東京)`
☒ライフサイクルルール (1年後自動削除) が設定される → 要件定義書 7.2 準拠

☒重要な用語

用語	意味
バケット	ストレージの「大きなフォルダ」。1プロジェクトに複数作れる
オブジェクト	バケット内のファイル本体 (画像1枚 = 1オブジェクト)
Standard Storage	標準ストレージ (今回はこれ)
ライフサイクル	「○日後に自動削除」等のルール

Step 1. Cloud Storage コンソールを開く

1. GCP Console でプロジェクトが `manabi-ops` になっていることを確認
2. 画面左メニュー (☰) → **「Cloud Storage」** → **「バケット」**
   - または上部検索バーで `Cloud Storage` と検索

Step 2. バケットを作成

1. 画面上部の **「+ 作成」** (CREATE) ボタンをクリック

2-1. バケット名

- **名前**: `manabi-ops-media`
- ⚠ **全世界でユニーク**である必要があります (既に他者が使っている場合は `manabi-ops-media-001` のように数字を追加)
- 使える文字: 小文字英数字・ハイフン・アンダースコア・ピリオド
- **「続行」** をクリック

2-2. データの保存場所

- ロケーションタイプ: **「Region」** (☒Region を選択 — Multi-region は高コスト)
- リージョン: `asia-northeast1 (Tokyo)` ☒東京
- **「続行」** をクリック

2-3. ストレージクラス

- **「Standard」** (標準) を選択 (デフォルト)
- ☒Nearline / Coldline / Archive は選ばない (アクセス頻度が低いデータ向け)
- **「続行」** をクリック

2-4. アクセス制御

- **「均一」** (Uniform) を選択 ☒
- ☒「きめ細かい管理」(Fine-grained) は選ばない
- **「公開アクセス禁止を適用する」** にチェック ☒(☒重要 — 画像は公開しない)
- **「続行」** をクリック

2-5. データの保護

- オブジェクトのバージョニング: **オフ** (OFF) のまま (コスト節約)
- 保持ポリシー: **設定しない**
- **「作成」** をクリック

2-6. 公開アクセス警告が出たら

- 「このバケットへの公開アクセスはブロックされます」と出たら **「確認」** をクリック

→ **バケットが作成されました** ☒

Step 3. ライフサイクルルール設定 (要件定義書 7.2 準拠)

要件定義書 7.2「メディアは1年で自動削除」を実現します。

1. 作成したバケット `manabi-ops-media` をクリック
2. 上部タブの **「ライフサイクル」** (Lifecycle) をクリック
3. **「+ ルールを追加」** (ADD A RULE) をクリック

ルール1: 1年後に削除

- **アクション**: **「オブジェクトを削除」** (Delete object)
- **「続行」** をクリック
- **オブジェクトの条件を選択**:
- ☑ **「年齢」** (Age) にチェック
- 値: `365` (日)
- **「作成」** をクリック

→ これで **1年経過したファイルは自動削除** されます (人手不要)

Step 4. フォルダ構造は自動作成される

ManabiOps は以下のフォルダ構造に自動でファイルを保存します。
手動作成は不要です。

```
manabi-ops-media/
├── receipts/          ← レシート画像（OCR前の原本）
├── photos/            ← イベント写真（オリジナル）
├── photos_blurred/    ← 顔ぼかし済み写真
├── videos/            ← 動画（リール素材）
├── videos_edited/     ← 編集済みリール動画
└── rosters/           ← 名簿画像
```

Step 5. IAMでサービスアカウントにアクセス権付与

08_secret_manager.md でサービスアカウントを作成した後、このバケットへの書込権限を与えます。
今この段階では何もしなくてOKです (後で説明します)。

☒完了チェック

- [ ] バケット名: `manabi-ops-media` (または `-001` 等)

- [ ] ロケーション: `asia-northeast1 (Tokyo)`
- [ ] ストレージクラス: Standard
- [ ] アクセス制御: 均一
- [ ] 公開アクセス禁止: 適用済み
- [ ] ライフサイクル: 365日後に削除するルールが設定済み

⌧メモしておく値

- `GCS_BUCKET_NAME` = `manabi-ops-media` (自分で設定した名前)

⌧次のステップ

→ **07_google_sheets.md** — 会計台帳スプレッドシート作成へ進む

🆘 トラブルシューティング

⌧「このバケット名は既に使用されています」

- 名前は **全世界で1つ** のため、`manabi-ops-media-2026` のように年号や数字を追加
- 最終的な名前を `.env` の `GCS_BUCKET_NAME` に設定する

⌧作成後、料金が心配

- 無料枠: **5GB/月 + 5000回の書込無料**
- ライフサイクル365日で自動削除するため無限に蓄積することはありません
- 月10GB程度のアップロードで **月¥15〜¥30** 程度

⌧誤ってバケットを削除してしまった

- バケット内に1ファイルもなければすぐに同名で再作成可能
- ファイルがあった場合、削除から最大30日は復旧可能な場合あり → GCP サポートへ

07. Google Sheets 会計台帳作成

所要時間: **15分**
費用: **¥0** (Google 無料サービス)
このガイドで取得するもの: `SHEETS_SPREADSHEET_ID`

☒このガイドでやること

ManabiOps は、LINE で承認されたレシート取引を **Google Sheets の会計台帳** に自動追記します。
これは要件定義書 **FR-03** (会計台帳自動記帳) を実現するためです。

ゴール

☒「まなびキッチン 会計台帳」スプレッドシートが作成される
☒1行目にヘッダー (日付・種別・勘定科目・金額・...) が入力される
☒サービスアカウントが「編集者」として共有される
☒ `SHEETS_SPREADSHEET_ID` をメモする

☒前提: なぜ Firestore ではなく Sheets か?

要件定義書 **第2章** より:

> 「運営代表者が PC が苦手で、データは **普段使い慣れた Excel/Google Sheets** で見たい」

そのため、会計データは **Firestore + Google Sheets の両方** に保存する設計になっています。
- **Firestore**: システムの内部データベース (高速・集計用)
- **Google Sheets**: 人間が見る・CSV出力する用

Step 1. 新しいスプレッドシートを作成

1. Google Sheets を開く
   - **運営代表者の Google アカウント** でログイン (他の人のアカウントでは☒
2. 左上の **「+ 空白」** (Blank) をクリック

1-1. ファイル名を変更

- 画面左上の **「無題のスプレッドシート」** をクリック
- 名前を **`まなびキッチン 会計台帳`** に変更 (日本語OK)
- Enterキーで確定

Step 2. シート名を変更

- 画面下部のタブ **「シート1」** をダブルクリック
- **`ledger`** に変更 (⌧半角英字で `ledger`)
- Enterキーで確定

> ⌧ `ledger` という名前は ManabiOps コード内で固定されているため、**必ず半角英字で `ledger`** にしてください。

Step 3. ヘッダー行を入力

1行目に以下の **10列** を入力します。

A	B	C	D	E	F	G	H	I	J
日付	種別	勘定科目	金額	店舗	品目	取引ID	承認者	承認日時	レシートURL

入力手順

1. A1 セルをクリック
2. 以下の1行を全コピー → A1 に貼り付け (タブ区切りで一度に入る)

```
日付	種別	勘定科目	金額	店舗	品目	取引ID	承認者	承認日時	レシートURL
```

1. ヘッダー行を太字に: A1:J1 を選択 → **Ctrl+B** (Macは ⌘+B)
2. ヘッダー行を固定: **「表示」** メニュー → **「固定」** → **「1行」**

Step 4. スプレッドシートIDを取得

1. 画面上部のURL欄を見る:

```
https://docs.google.com/spreadsheets/d/1AbCdEfGhIjKlMnOpQrStUvWxYz0123456789/edit
                                       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
                                       これが SPREADSHEET_ID
```

1. URL の `/d/` と `/edit` の間の長い文字列 (約44文字) をコピー
2. `SHEETS_SPREADSHEET_ID` としてメモしておく (`.env` で使用)

☒メモ例

```
SHEETS_SPREADSHEET_ID=1AbCdEfGhIjKlMnOpQrStUvWxYz0123456789
```

Step 5. サービスアカウントと共有

02_gcp_project.md Step 5 で作成したサービスアカウントメールアドレスに、
このスプレッドシートへの編集権限を付与します。

5-1. サービスアカウントのメールアドレスを確認

- 形式: `manabi-ops-sa@manabi-ops.iam.gserviceaccount.com`
- GCP Console → **IAM と管理** → **サービス アカウント** で確認できる
- **このメールアドレスをコピー**

5-2. スプレッドシートで共有

1. スプレッドシート右上の **「共有」** ボタン (青色) をクリック
2. 「ユーザーやグループを追加」欄に、**サービスアカウントのメールアドレス** を貼り付け
3. 右側の権限が **「編集者」** (Editor) であることを確認
4. **「通知を送信」のチェックを外す** (☒サービスアカウントにメールは届かないため)
5. 「ロールを確認」の警告が出たら **「通知なしで共有」** (Share anyway) をクリック
6. **「完了」** をクリック

Step 6. (任意) 書式を見やすく整える

運営代表者が見やすいように、お好みで以下を設定:

- **金額列 (D列)** を「通貨」形式に:
- D列ヘッダー (D) を右クリック → **「列全体を選択」** → **「表示形式」** → **「数値」** → **「通貨」** → **¥**
- **日付列 (A列)** を「2026/04/16」形式に:
- A列ヘッダー → **「表示形式」** → **「数値」** → **「日付」**

- **ヘッダー行の背景色**:
- A1:J1 選択 → 塗りつぶしアイコン → 薄い黄色など
- **行の縞模様**:
- A1:J1000 選択 → **「表示形式」** → **「交互の背景色」** → **OK**

Step 7. 集計シート (任意・便利)

月次集計を手軽に見たい場合、もう1つシートを追加します。

1. 画面下部の **「+」** アイコンで新しいシートを追加
2. シート名を `summary` に変更
3. 以下のセルに関数を入力:

セル	関数	意味
A1	`月`	ヘッダー
B1	`収入合計`	ヘッダー
C1	`支出合計`	ヘッダー
D1	`残高`	ヘッダー
A2	`=TEXT(TODAY(),"yyyy/MM")`	今月
B2	`=SUMIFS(ledger!D:D, ledger!B:B, "収入", ledger!A:A, ">="&EOMONTH(TODAY(),-1)+1)`	今月の収入
C2	`=SUMIFS(ledger!D:D, ledger!B:B, "支出", ledger!A:A, ">="&EOMONTH(TODAY(),-1)+1)`	今月の支出
D2	`=B2-C2`	今月の残高

→ ManabiOps がデータを追記する度に自動で集計されます。

☒完了チェック

- [ ] スプレッドシート名: `まなびキッチン 会計台帳`
- [ ] シート名: `ledger` (半角英字)
- [ ] ヘッダー10列が入力済み (日付〜レシートURL)
- [ ] サービスアカウントが **編集者** として共有済み
- [ ] `SHEETS_SPREADSHEET_ID` をメモ済み

⊠次のステップ

→ **08_secret_manager.md** — Secret Manager に全キーを登録へ進む

🆘 トラブルシューティング

⊠サービスアカウントへの共有でエラー

- メールアドレスのコピー間違いがないか確認 (`@manabi-ops.iam.gserviceaccount.com` で終わる形式)
- 「通知を送信」のチェックを外すことを忘れずに

⊠スプレッドシートIDが分からない

- URL全体を運営代表者・開発担当者に送れば読み取れます
- 形式: `https://docs.google.com/spreadsheets/d/【ここがID】/edit`

⊠ManabiOps デプロイ後、データが書き込まれない

- サービスアカウントの共有権限が「編集者」になっているか
- Google Sheets API が有効か (02_gcp_project.md Step 4 参照)
- デプロイ後の Cloud Run ログを 13_troubleshooting.md で確認

08. Secret Manager にシークレット登録

所要時間: **20分**
費用: **月6シークレットまで無料** → **¥0** 想定
このガイドで取得するもの: Cloud Run がシークレットを読み込める状態

☒このガイドでやること

これまでのガイドで取得した **API キー・トークン等の機密情報** を、
GCP の金庫 (Secret Manager) に安全に保存します。

ゴール

☒以下のシークレットがすべて登録される

#	シークレット名	値	どこで取得したか
1	`line-channel-secret`	LINE Channel Secret	01
2	`line-channel-access-token`	LINE アクセストークン	01
3	`gemini-api-key`	Gemini API キー	03
4	`docai-receipt-processor-id`	レシートプロセッサID	04
5	`docai-roster-processor-id`	名簿プロセッサID (Phase 2)	04
6	`sheets-spreadsheet-id`	Google Sheets ID	07
7	`encryption-key`	氏名暗号化用32バイトキー (新規生成)	このガイドで作成
8	`admin-basic-auth`	管理画面用 Basic認証	このガイドで作成
9	`ig-access-token` (Phase 3)	Instagram アクセストークン	12
10	`ig-business-account-id` (Phase 3)	Instagram ビジネスアカウントID	12

☒サービスアカウントにシークレット読取権限 (Secret Manager Secret Accessor) が付与される

☒なぜ Secret Manager を使うのか

要件定義書 **5.3 セキュリティ要件** より:

> 「APIキー・トークンは Secret Manager に保管し、ソースコードにベタ書きしない」

- ☒ `.env` ファイルにキーを書いて GitHub にアップロードする → 情報漏洩
- ☒Secret Manager にキーを保管し、Cloud Run 実行時に安全に読み込み

Step 1. Secret Manager を有効化

1. GCP Console → プロジェクト `manabi-ops` を選択
2. 上部検索バーで `Secret Manager` を検索 → 開く
3. 「API を有効にする」画面が出たら **「有効にする」** をクリック (初回のみ)

Step 2. シークレットを登録する共通手順

各シークレットで以下の手順を **繰り返します**。

共通手順

1. Secret Manager 画面上部の **「+ シークレットを作成」** をクリック
2. **名前** に上記表の「シークレット名」を入力 (例: `line-channel-secret`)
3. **シークレットの値** に実際の値 (例: LINE で取得した Channel Secret) を貼り付け
   - ⚠ 前後に空白や改行が入らないよう注意
4. その他の項目はデフォルトのまま
5. **「シークレットを作成」** をクリック

Step 3. 各シークレットを登録

① `line-channel-secret`

- 値: 01_line_messaging_api.md で取得した **Channel Secret** (32文字)

② `line-channel-access-token`

- 値: 01_line_messaging_api.md で取得した **Channel Access Token** (長文字列)

③ `gemini-api-key`

- 値: 03_gemini_api.md で取得した `AIza...` で始まるキー

④ `docai-receipt-processor-id`

- 値: 04_document_ai.md で取得した **レシート用プロセッサID** (例: `abc123...`)

⑤ `docai-roster-processor-id` (Phase 2 以降)

- 値: 04_document_ai.md の **カスタムエクストラクタ (名簿) のID**
- Phase 2 まで未取得の場合は、仮に空文字 `""` で登録 (後で更新)

⑥ `sheets-spreadsheet-id`

- 値: 07_google_sheets.md で取得した **スプレッドシートID** (44文字)

⑦ `encryption-key` (新規生成)

要件定義書 **12章** の「参加者氏名の暗号化」に使います。以下の手順で **新規生成** してください。

生成方法 (Mac/Linux)

ターミナルで以下を実行:

```
openssl rand -base64 32
```

出力される44文字の文字列 (例: `Kj8Fg9LmPq7Rs2Tu4VwX6YZ0aB1cD3eF5G7hI9jK=`) をコピー。

生成方法 (Windows)

PowerShell で以下を実行:

```
[Convert]::ToBase64String((1..32 | %{[byte](Get-Random -Max 256)}))
```

生成方法 (オンライン)

https://generate-random.org/encryption-key-generator → **256-bit** → **Base64** で生成

- ⚠ **このキーは絶対に再生成しない** (過去の参加者データが復号できなくなる)
- ⚠ キーを別途オフライン (USB等) にもバックアップ

⑧ `admin-basic-auth`

管理画面 (`/admin`) のBasic認証用。形式: `ユーザー名:パスワード`

- 値: `admin:【強固なパスワード16文字以上】`
- 例: `admin:Mk8pQ2xR9vN4zT7wL3y`
- パスワードは自分で決める (パスワードマネージャで生成推奨)

⑨ `ig-access-token` (Phase 3)

- 値: 12_instagram_graph_api.md で取得
- Phase 3 実施まで未取得の場合、このシークレット登録はスキップ可

⑩ `ig-business-account-id` (Phase 3)

- 値: 12_instagram_graph_api.md で取得
- Phase 3 まで未取得の場合、スキップ可

Step 4. サービスアカウントに読取権限を付与

Cloud Run がこれらのシークレットを読めるように、
サービスアカウントに権限を付与します。

方法 A: 一括で全シークレットに権限付与 (推奨)

1. GCP Console → **IAM と管理** → **IAM**
2. **「+ アクセスを付与」** をクリック
3. 「新しいプリンシパル」にサービスアカウントのメールアドレスを入力:
   `manabi-ops-sa@manabi-ops.iam.gserviceaccount.com`
4. 「ロール」に `Secret Manager Secret Accessor` を選択
5. **「保存」**

方法 B: 各シークレットに個別付与

各シークレットを開いて「権限」タブから付与する方法もありますが、
方法Aの方が簡単で一貫性があります。

Step 5. 設定確認

1. Secret Manager 画面で、以下のシークレットが **すべて** 存在することを確認:

✓	シークレット名
☐	line-channel-secret
☐	line-channel-access-token
☐	gemini-api-key
☐	docai-receipt-processor-id
☐	docai-roster-processor-id (またはスキップ)
☐	sheets-spreadsheet-id
☐	encryption-key
☐	admin-basic-auth
☐	ig-access-token (またはスキップ)
☐	ig-business-account-id (またはスキップ)

1. いずれかのシークレットをクリック → **「バージョン」** タブに「Active」の行が1件以上あれば正常

☒完了チェック

- [ ] Secret Manager が有効化された
- [ ] Phase 1 必須の6シークレットが登録された
- [ ] `encryption-key` がオフラインバックアップされた
- [ ] サービスアカウントに `Secret Manager Secret Accessor` 権限付与済み

☒次のステップ

→ **09_cloud_run_deploy.md** — Cloud Run にアプリをデプロイへ進む

🆘 トラブルシューティング

☒シークレット値を間違えて登録した

1. 該当シークレットを開く
2. **「+ 新しいバージョン」** をクリック
3. 正しい値を入力 → 保存

4. 古いバージョンは「無効化」できるが、新バージョンが自動で使われるため通常放置でOK

☒サービスアカウントのメールアドレスが分からない

- GCP Console → **IAM と管理** → **サービス アカウント**
- 一覧から `manabi-ops-sa` を探して、メールアドレスをコピー

☒Secret Manager APIが有効化できない

- 請求アカウントがリンクされているか 02_gcp_project.md で確認
- プロジェクトオーナー権限が自分のGoogleアカウントにあるか確認

☒ `encryption-key` を紛失した

- 既に暗号化した参加者氏名データが **永久に復号できなくなる** (⚠ 重大)
- 初期段階 (データが少ない) であれば、新キー生成 + Firestore `participants` コレクション削除で復旧可
- データ移行期の場合は、開発担当者に至急連絡

09. Cloud Run にアプリをデプロイ

所要時間: **30分**
費用: **月200万リクエスト無料** → まなびキッチン規模では **¥0〜¥300**
このガイドで取得するもの: Cloud Run サービスURL (例: `https://manabi-ops-xxxxx.a.run.app`)

☒このガイドでやること

ManabiOps のソースコードを Cloud Run (GCP のサーバーレス実行環境) にデプロイし、
24時間365日稼働するサーバー を立ち上げます。

ゴール

☒Cloud Run サービス `manabi-ops` が稼働
☒HTTPS URL が発行される
☒すべてのシークレットが自動で読み込まれる
☒ `/healthz` にアクセスして `{"ok":true, "mode":"production"}` が返る

☒デプロイ方法は2種類

方法	難易度	おすすめ度
A. Cloud Build + GitHub 連携 (自動)	☒☒	継続開発する場合に最適
B. gcloud CLI で手動デプロイ (1回のみ)	☒	とりあえず動かしたい場合

本ガイドでは **方法 B** を中心に解説します (最短)。方法Aは末尾でオプションとして紹介。

方法 B: gcloud CLI で手動デプロイ

Step 1. gcloud CLI をインストール

Mac / Windows / Linux のいずれでも同じ手順。

インストール手順

1. Google Cloud CLI インストーラ を開く
2. OS を選択して、公式手順通りにインストール
3. インストール後、ターミナル (Mac/Linux) または PowerShell (Windows) を開く
4. 以下で動作確認:

```
gcloud --version
```

→ `Google Cloud SDK 4xx.x.x` のような表示が出ればOK

Step 2. gcloud を初期化

```
gcloud auth login
```

→ ブラウザが開く → **運営代表者の Google アカウント** でログイン → 許可

```
gcloud config set project manabi-ops
```

→ `Updated property [core/project].` と表示されればOK

```
gcloud config set run/region asia-northeast1
```

→ デフォルトリージョンを東京に設定

Step 3. ソースコードを取得

```
# 任意の作業ディレクトリで
git clone https://github.com/satoshiyoshimoto0426/manabi-kitchin.git
cd manabi-kitchin
```

Step 4. 必要な API を有効化

以下を1つずつ実行 (既に有効化済みなら「already enabled」と出るだけ):

```
gcloud services enable run.googleapis.com
gcloud services enable cloudbuild.googleapis.com
gcloud services enable secretmanager.googleapis.com
gcloud services enable artifactregistry.googleapis.com
gcloud services enable firestore.googleapis.com
gcloud services enable storage.googleapis.com
gcloud services enable documentai.googleapis.com
gcloud services enable sheets.googleapis.com
gcloud services enable aiplatform.googleapis.com
```

Step 5. Cloud Run にデプロイ

以下の **1コマンド** で、ビルド → イメージ保存 → デプロイ まで自動実行されます。

```
gcloud run deploy manabi-ops \
  --source . \
  --region asia-northeast1 \
  --platform managed \
  --allow-unauthenticated \
  --service-account manabi-ops-sa@manabi-ops.iam.gserviceaccount.com \
  --memory 1Gi \
  --cpu 1 \
  --timeout 300 \
  --max-instances 3 \
  --min-instances 0 \
  --set-env-vars "NODE_ENV=production,MOCK_MODE=false,GCP_PROJECT_ID=manabi-ops,GCS_BUCKET_NAME=manabi-ops-media,GCP_LOCATION=asia-northeast1" \
  --set-secrets "LINE_CHANNEL_SECRET=line-channel-secret:latest,LINE_CHANNEL_ACCESS_TOKEN=line-channel-access-token:latest,GEMINI_API_KEY=gemini-api-key:latest,DOCAI_RECEIPT_PROCESSOR_ID=docai-receipt-processor-id:latest,DOCAI_ROSTER_PROCESSOR_ID=docai-roster-processor-id:latest,SHEETS_SPREADSHEET_ID=sheets-spreadsheet-id:latest,ENCRYPTION_KEY=encryption-key:latest,ADMIN_BASIC_AUTH=admin-basic-auth:latest"
```

各オプションの意味

オプション	意味
`--source .`	カレントディレクトリのソースをビルド
`--allow-unauthenticated`	LINE/Instagram の Webhook が匿名アクセスできるよう許可
`--service-account`	02 で作ったサービスアカウント
`--memory 1Gi`	顔ぼかし処理のためメモリ1GB
`--max-instances 3`	同時稼働の最大数 (コスト抑制)
`--min-instances 0`	使用がない時は0台 (無料)
`--set-env-vars`	環境変数 (シークレット以外)
`--set-secrets`	Secret Manager から自動読込

デプロイにかかる時間

初回: **5〜10分** (Docker イメージのビルド含む)
2回目以降 (コード更新時): **2〜5分**

Step 6. Phase 3 (Instagram連携) を有効化する場合

Phase 3 で Instagram 投稿を有効にする場合は、Step 5 のコマンドに以下を **追加**:

```
,IG_ACCESS_TOKEN=ig-access-token:latest,IG_BUSINESS_ACCOUNT_ID=ig-business-account-id:latest
```

(カンマ忘れに注意)

Step 7. デプロイ成功を確認

7-1. URL を確認

デプロイ成功時、以下のような出力が出ます:

```
Service [manabi-ops] revision [manabi-ops-00001-abc] has been deployed
and is serving 100 percent of traffic.
Service URL: https://manabi-ops-xxxxxx-an.a.run.app
```

この Service URL をコピー・メモ (後で LINE Webhook に設定)

7-2. ヘルスチェック

```
curl https://manabi-ops-xxxxxx-an.a.run.app/healthz
```

期待される出力:

```
{
  "ok": true,
  "mode": "production",
  "services": {
    "line": true,
    "gemini": true,
    "docai": true,
    "firestore": true,
    "sheets": true,
    "gcs": true,
    "ig": false
  },
  "time": "2026-04-16T..."
}
```

- `mode: production` になっていればOK
- `services.*: true` の数が多いほど正しく接続できている
- `ig: false` は Phase 3 未実施なら正常

7-3. 管理画面にアクセス

ブラウザで:

```
https://manabi-ops-xxxxxx-an.a.run.app/admin
```

→ Basic認証を要求される → 08_secret_manager.md の `admin-basic-auth` で設定した **ユーザー名:パスワード** を入力 → 管理ダッシュボードが表示されればOK

🆘 Step 5 でエラーが出た場合

❌ `PERMISSION_DENIED`

- サービスアカウントに `Cloud Run 管理者` ロールが付与されているか確認
- または自分のGoogleアカウントにプロジェクトオーナー権限があるか確認

❌ `Secret not found`

- 08_secret_manager.md で登録漏れがないか確認
- シークレット名の綴りを確認 (`line-channel-secret` 等、ハイフン区切り)

⊠ビルド失敗

- `node_modules` を含めていないか確認 → `.dockerignore` で除外済みのはず
- Dockerfile がルートディレクトリにあるか確認

⊠メモリ不足 (顔ぼかし処理)

- `--memory 1Gi` を `--memory 2Gi` に増やす (わずかに料金増)

⊠次のステップ

→ **10_line_webhook.md** — LINE Webhook URL を設定へ進む

方法 A: GitHub 連携による自動デプロイ (上級者向け)

コードを更新するたびに `git push` で自動的に Cloud Run にデプロイされる仕組みです。

概要手順

1. GCP Console → **Cloud Run** → 対象サービス `manabi-ops` を開く
2. **「編集とデプロイの新しいリビジョン」** → **「継続的デプロイ」** を設定
3. GitHub リポジトリ `satoshiyoshimoto0426/manabi-kitchin` と連携
4. ブランチ `main` にマージされたら自動デプロイ

詳細は Cloud Run 公式ドキュメント を参照。

☒完了チェック

- [ ] `gcloud run deploy` が成功
- [ ] `/healthz` が `{"ok":true,"mode":"production"}` を返す
- [ ] `/admin` にログイン可能
- [ ] Service URL をメモ済み

10. LINE Webhook URL を設定

> 所要時間: **10分**
> 費用: **¥0**
> 前提: 09_cloud_run_deploy.md で Cloud Run Service URL を取得済み

☒このガイドでやること

Cloud Run にデプロイされた ManabiOps の Webhook URL を
LINE 側に登録し、LINEからのメッセージがサーバーに届くようにします。

ゴール

☒LINE Developers Console で Webhook URL 登録完了
☒「検証」ボタンで200 OK 応答を確認
☒実機の LINE 公式アカウントに画像を送り、自動返信が来る

Step 1. Webhook URL を準備

09_cloud_run_deploy.md で取得した Cloud Run の Service URL に
`/webhook` を追加したものが Webhook URL になります。

例

- Cloud Run URL: `https://manabi-ops-xxxxxx-an.a.run.app`
- Webhook URL: **`https://manabi-ops-xxxxxx-an.a.run.app/webhook`**

→ この URL をコピー

Step 2. LINE Developers Console で設定

2-1. コンソールを開く

1. LINE Developers Console を開く
2. 01_line_messaging_api.md で作成した **Provider** を選択

3. **「まなびキッチン」** の Messaging API チャネルを開く

2-2. Webhook URL を設定

1. 上部タブの **「Messaging API 設定」** (Messaging API) をクリック
2. 下にスクロールして **「Webhook 設定」** セクションを探す
3. **「Webhook URL」** 欄に、Step 1 でコピーした URL を貼り付け
4. **「更新」** (Update) をクリック

2-3. Webhook を有効化

1. 「Webhook の利用」(Use webhook) のトグルを **オン** (有効) に
2. すぐ右の **「検証」** (Verify) ボタンをクリック

2-4. 検証結果

- ☒ **「成功」** (Success) と表示されればOK
- ☒エラーが出た場合は末尾のトラブルシューティングへ

Step 3. 自動応答メッセージを無効化

LINE は既定で「ありがとうございます…」のような自動応答メッセージを返します。
これがManabiOps の返信と干渉するため **無効化** します。

1. 同じ「Messaging API 設定」ページ内、 **「LINE 公式アカウント機能」** セクションを探す
2. **「応答メッセージ」** → **「編集」** をクリック
   → LINE Official Account Manager が別タブで開く
3. 左メニュー → **「応答設定」** を開く
4. 以下のように設定:
項目	設定
チャット	**オフ**
あいさつメッセージ	お好み (オフ推奨)
応答メッセージ	**オフ**
Webhook	**オン** ☒
5. 保存

Step 4. 初回ユーザー登録 (運営代表者)

ManabiOps は **「最初にメッセージを送った LINE ユーザーを自動で Owner 権限として登録」** する仕様になっています。
これは要件定義書 EX-10 対応 + 運用開始時の便宜のためです。

手順

1. 自分のスマホで LINE アプリを開く
2. まなびキッチンの **LINE 公式アカウント** を友達追加
   - QRコードは LINE Developers Console の「Messaging API 設定」→ 下部の QR で表示可
3. **任意のメッセージを送る** (例: `こんにちは`)
4. ManabiOps からの返信が来るか確認

期待される返信

- 初回: `"☒ようこそ ManabiOps へ！運営代表者として登録しました。"`
- 2回目以降: コマンドヘルプ等

Step 5. 動作確認: レシート画像を送る

1. 手元のレシート (またはネットで拾ったサンプル画像) を LINE で送信
2. **30秒以内** に ManabiOps から Flex Message (確認カード) が返ってくるはず

期待される Flex Message

```
────────────────────
レシート確認
────────────────────
○○スーパー
2026/04/16
¥4,820

勘定科目：食材費
品目：米5kg，人参3本，...

[☒登録] [✏ 修正] [☒取消]
────────────────────
```

→ **[☒登録]** をタップすると Google Sheets に自動追記される

☒完了チェック

- [ ] Webhook URL が LINE Developers Console に登録された
- [ ] 「検証」で成功が返った
- [ ] 応答メッセージが無効化された
- [ ] 自分 (運営代表者) が初回ログインで Owner 権限を取得できた
- [ ] レシート画像を送って Flex Message が返ってきた
- [ ] 承認すると Google Sheets に記帳された

☒次のステップ

→ **11_cloud_scheduler.md** — Cloud Scheduler (1時間バッチ) 設定へ進む

🆘 トラブルシューティング

☒検証エラー: `The webhook returned an HTTP status code other than 200`

- Cloud Run URL が正しいか (`/webhook` を忘れていないか)
- Cloud Run が起動しているか: `curl YOUR_URL/healthz` で確認
- `--allow-unauthenticated` オプションでデプロイされているか (09 Step 5)

☒検証エラー: `The signature in the webhook request header was invalid`

- Secret Manager の `line-channel-secret` の値が正しいか再確認
- 前後の空白・改行を含めていないか

☒メッセージを送っても返事が来ない

1. **応答メッセージ** がオフになっているか
2. **Webhook** がオンになっているか
3. Cloud Run のログを確認:

```
gcloud run services logs read manabi-ops --region asia-northeast1 --limit 50
```

1. エラーが出ている場合 → 13_troubleshooting.md

☒「☒権限がありません」と返ってきた

- 2人目以降のユーザーはホワイトリスト登録が必要
- 管理画面 (`/admin`) にログインして、**ユーザー管理** から LINE User ID を追加

- または Owner ユーザーが LINE でコマンド `/adduser スタッフ名 U_xxxx` を実行

11. Cloud Scheduler (1時間バッチ) 設定

所要時間: **15分**
費用: **月3ジョブまで無料 → ¥0**
このガイドで取得するもの: 1時間ごとの自動バッチ実行環境

☒このガイドでやること

ManabiOps は以下の **定期タスク** を裏で回す必要があります (要件定義書 EX-05, EX-07, EX-11)。

ジョブ	頻度	内容
hourly-batch	毎時	承認待ち・リマインダー確認、期限切れ自動取消、ストレージ容量確認
monthly-summary	月初1日 9:00	前月の会計・参加者集計を Owner に LINE 送信 (FR-11)
daily-cleanup	毎日 3:00	古い承認待ちデータ、一時ファイル削除

Cloud Scheduler は GCP のタスクスケジューラ (cron) です。

ゴール

☒3つのジョブが作成される
☒すべて `manabi-ops` Cloud Run サービスの `/internal/*` エンドポイントを呼び出す
☒認証トークン付きで実行される (サービスアカウント経由)

Step 1. Cloud Scheduler を有効化

1. GCP Console → プロジェクト `manabi-ops` を選択
2. 上部検索バーで `Cloud Scheduler` を検索 → 開く
3. 「API を有効にする」画面が出たら **「有効にする」** をクリック (初回のみ)

Step 2. 共通: 認証用サービスアカウントの準備

Cloud Scheduler が Cloud Run を呼び出すのに、認証トークンが必要です。
02_gcp_project.md で作成したサービスアカウント `manabi-ops-sa` を使います。

権限確認

サービスアカウントに **「Cloud Run 起動元」** (`roles/run.invoker`) が付与されているか確認:

1. GCP Console → **IAM と管理** → **IAM**
2. `manabi-ops-sa@manabi-ops.iam.gserviceaccount.com` の行を探す
3. ロール欄に **「Cloud Run 起動元」** が含まれていればOK
4. 無ければ **編集 (鉛筆アイコン)** → **「+ ロールを追加」** → `Cloud Run Invoker` を追加 → 保存

Step 3. ジョブ① hourly-batch (毎時実行)

1. Cloud Scheduler 画面上部の **「+ ジョブを作成」** をクリック

3-1. 基本設定

項目	値
名前	`manabi-ops-hourly-batch`
リージョン	`asia-northeast1 (東京)`
説明	`ManabiOps 毎時バッチ (EX-05/07/11)`
頻度	`0 * * * *` (毎時0分)
タイムゾーン	`Asia/Tokyo` (日本標準時)

「続行」 をクリック

3-2. 実行内容

項目	値
ターゲット タイプ	**HTTP**
URL	`https://manabi-ops-xxxxxx-an.a.run.app/internal/hourly` (自分のCloud Run URL)
HTTP メソッド	**POST**
HTTP ヘッダー	`Content-Type: application/json`
本文	`{}`
Auth ヘッダー	**「OIDC トークンを追加」**
サービスアカウント	`manabi-ops-sa@manabi-ops.iam.gserviceaccount.com`
対象 (audience)	Cloud Run URL (`https://manabi-ops-xxxxxx-an.a.run.app`)

「続行」 をクリック

3-3. 再試行設定 (任意)

- 最大再試行回数: **3**
- 最大再試行時間: **10分**
- **「作成」** をクリック

Step 4. ジョブ② monthly-summary (月初実行)

同じ手順で2つ目のジョブを作成。

4-1. 基本設定

項目	値
名前	`manabi-ops-monthly-summary`
頻度	`0 9 1 * *` (毎月1日 9:00)
タイムゾーン	`Asia/Tokyo`

4-2. 実行内容

項目	値
URL	`https://manabi-ops-xxxxxx-an.a.run.app/internal/monthly-summary`
HTTP メソッド	**POST**
本文	`{}`
Auth	OIDCトークン (同じサービスアカウント)

Step 5. ジョブ③ daily-cleanup (毎日3時)

5-1. 基本設定

項目	値
名前	`manabi-ops-daily-cleanup`
頻度	`0 3 * * *` (毎日3:00)
タイムゾーン	`Asia/Tokyo`

5-2. 実行内容

項目	値
URL	`https://manabi-ops-xxxxxx-an.a.run.app/internal/daily-cleanup`
HTTP メソッド	**POST**
本文	`{}`
Auth	OIDCトークン (同じサービスアカウント)

Step 6. 即時テスト実行

各ジョブが正しく動くかを、次回実行を待たずに確認します。

1. Cloud Scheduler 画面でジョブ一覧を表示
2. 対象ジョブの行 → 右端の **「⋮」(縦3点)** → **「今すぐ実行」**
3. 数秒後に **「最終実行結果」** が **「成功」** になればOK

Step 7. ログ確認

1. ジョブ行のメニュー (縦3点) → **「ログを表示」**
2. 実行ログが並んでいるか確認
3. HTTPステータスが **200** になっているかチェック

Cloud Run 側のログも確認:

```
gcloud run services logs read manabi-ops --region asia-northeast1 --limit 20
```

Step 8. cron 式早見表

やりたいこと	cron式
毎時0分	`0 * * * *`
毎時30分	`30 * * * *`
毎日9:00	`0 9 * * *`
毎日0:00と12:00	`0 0,12 * * *`
平日9:00	`0 9 * * 1-5`
月初1日9:00	`0 9 1 * *`
月末9:00	`0 9 L * *`

詳細: cron 書式

☒完了チェック

- [ ] Cloud Scheduler API 有効化
- [ ] `manabi-ops-hourly-batch` ジョブ作成 (毎時0分)
- [ ] `manabi-ops-monthly-summary` ジョブ作成 (月初1日 9:00)
- [ ] `manabi-ops-daily-cleanup` ジョブ作成 (毎日3:00)
- [ ] 各ジョブを「今すぐ実行」でテスト → 成功
- [ ] OIDC トークン認証が設定されている

☒次のステップ

→ **Phase 3 (Instagram投稿) に進む場合**: 12_instagram_graph_api.md
→ **動作確認・トラブル対応**: 13_troubleshooting.md

Phase 1 MVP だけ使う場合、ここでセットアップ完了です！ ☒

🆘 トラブルシューティング

☒「今すぐ実行」の結果が「失敗」

- HTTPステータスをログで確認

- **401 Unauthorized**: OIDC トークン設定が間違っている。サービスアカウントと audience を再確認
- **404 Not Found**: URL が間違い。`/internal/hourly` などのパスをチェック
- **500 Internal Server Error**: Cloud Run 側のエラー → `gcloud run services logs read` で詳細確認

☒ジョブが勝手に止まる

- 最大再試行回数に達した可能性 → Cloud Scheduler で「一時停止解除」
- Cloud Run がクラッシュしていないか `/healthz` で確認

☒二重実行される

- Cloud Scheduler の再試行時のタイムアウト設定を長めに (10分以上)
- ManabiOps 側はidempotent (重複実行しても安全) に設計済みなので通常は問題なし

12. Instagram Graph API 連携 (Phase 3)

所要時間: **90分** (Meta の審査待ち時間除く)
費用: **¥0**
このガイドで取得するもの: `IG_ACCESS_TOKEN` , `IG_BUSINESS_ACCOUNT_ID`
対象: **Phase 3 (SNS自動投稿) を実施する場合のみ**

☒このガイドでやること

要件定義書 **FR-10** (Instagram 自動投稿) を実現します。

ゴール

☒まなびキッチン用の **Facebook ページ** が作成される
☒Instagram アカウントが **ビジネス (or クリエイター) アカウント** に変換される
☒Facebook ページと Instagram が紐付けられる
☒Meta for Developers にアプリが登録される
☒**長寿命アクセストークン** (60日) が取得される
☒**ビジネスアカウントID** が取得される

☒全体フロー

```
[1] Facebook Page 作成
  ↓
[2] Instagram を Business アカウント化
  ↓
[3] Instagram と Facebook Page を紐付け
  ↓
[4] Meta for Developers にアプリ登録
  ↓
[5] Graph API Explorer でアクセストークン取得
  ↓
[6] 短期トークン → 長期トークン（60日）に変換
  ↓
[7] ビジネスアカウントID 取得
  ↓
[8] Secret Manager に登録
```

Part 1: Facebook Page 作成

Step 1. Facebook 個人アカウントの準備

- まなびキッチン運営代表者の個人 Facebook アカウントにログイン
- 持っていない場合: https://www.facebook.com/ で新規作成
- 2段階認証を **必ず有効化** (セキュリティのため)

Step 2. Facebook Page (ビジネスページ) 作成

1. https://www.facebook.com/pages/create にアクセス
2. ページ名: **「まなびキッチン」**
3. カテゴリ: **「コミュニティ組織」** または **「非営利団体」**
4. 詳細説明: 「◯◯市の子ども食堂 まなびキッチン」等
5. **「ページを作成」** をクリック
6. プロフィール画像・カバー画像は後で設定OK

Part 2: Instagram を Business 化

Step 3. Instagram アプリをインストール (まだの場合)

- スマホで Instagram アプリをダウンロード
- 「まなびキッチン」専用アカウントを作成
- ユーザーネーム: `manabi_kitchen` 等 (既に使用済みなら別名)

Step 4. ビジネスアカウントへ切替

1. Instagram アプリで **「設定とプライバシー」** → **「アカウントの種類とツール」** (または「プロアカウントに切替」)
2. **「プロアカウントに切り替える」** を選択
3. カテゴリ: **「コミュニティ」** または **「非営利団体」**
4. **「ビジネス」** を選択 (クリエイターより API 機能が豊富)
5. 画面の指示に従って完了

Step 5. Instagram と Facebook Page を紐付け

1. Instagram アプリで **「設定とプライバシー」** → **「アカウントセンター」**
2. **「アカウントを追加」** → Facebook を選択
3. Step 2 で作成した Facebook Page と連携

または PC で:
1. Facebook Page (まなびキッチン) を開く
2. 左メニュー **「設定」** → **「Instagram」**
3. **「アカウントを接続」** → Instagram ログイン → 連携完了

Part 3: Meta for Developers アプリ登録

Step 6. Meta 開発者アカウント登録

1. https://developers.facebook.com/ にアクセス
2. 右上 **「スタート」** or **「ログイン」** → 個人 Facebook アカウントでログイン
3. 開発者登録画面が出たら:
   - 電話番号認証
   - 利用規約に同意
4. 開発者アカウント完成

Step 7. アプリを作成

1. https://developers.facebook.com/apps/ を開く
2. **「アプリを作成」** (Create App) をクリック
3. 「ユースケースを選択」: **「その他」** (Other)
4. 「アプリタイプを選択」: **「ビジネス」** (Business)
5. アプリ名: `ManabiOps`
6. メールアドレス: 運営代表者のメール
7. **「アプリを作成」** をクリック

Step 8. Instagram Graph API 製品を追加

1. アプリのダッシュボードで、左メニュー **「製品」** → **「+ 製品を追加」**

2. 以下を検索して **「設定」** をクリック:
   - **「Instagram Graph API」** (または「Instagram API with Instagram Login」)
3. 次に **「Facebook ログイン」** も製品追加 (必要)

Part 4: アクセストークン取得

Step 9. Graph API Explorer でトークン取得

1. https://developers.facebook.com/tools/explorer/ を開く
2. 画面右側:
   - **「Meta App」**: `ManabiOps` を選択
   - **「User or Page」**: **「Get User Access Token」** を選択
3. **アクセス権限 (Permissions)** を追加:
   - ☑ `instagram_basic`
   - ☑ `instagram_content_publish`
   - ☑ `pages_show_list`
   - ☑ `pages_read_engagement`
   - ☑ `business_management`
4. **「Generate Access Token」** をクリック
5. Facebook ログイン画面が出る → 権限を全て許可
6. 画面に **短期トークン (2時間有効)** が表示される → コピー

Step 10. 長期トークン (60日) に変換

Meta で以下の URL を生成してブラウザで開く:

```
https://graph.facebook.com/v18.0/oauth/access_token?
  grant_type=fb_exchange_token&
  client_id=【APP_ID】&
  client_secret=【APP_SECRET】&
  fb_exchange_token=【短期トークン】
```

各値の取得場所

- `APP_ID`: アプリダッシュボード左メニュー **「設定」** → **「基本」**
- `APP_SECRET`: 同じく「基本」設定内。「表示」ボタンでパスワード入力して表示
- `短期トークン`: Step 9 でコピーしたもの

結果

JSON形式で返ってくる:

```
{
  "access_token": "EAAxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx",
  "token_type": "bearer",
  "expires_in": 5183944
}
```

→ この `access_token` が **`IG_ACCESS_TOKEN`** (60日有効)

Step 11. ビジネスアカウントID 取得

11-1. Page ID を取得

Graph API Explorer で以下を実行:

```
GET /me/accounts
```

→ 配列が返り、各要素に `id` (Page ID) と `name` (ページ名) がある
→ 「まなびキッチン」の `id` をコピー

11-2. Instagram Business Account ID を取得

```
GET /【PAGE_ID】?fields=instagram_business_account
```

→ レスポンス:

```
{
  "instagram_business_account": {
    "id": "17841xxxxxxxxxxx"
  },
  "id": "【PAGE_ID】"
}
```

→ `instagram_business_account.id` が **`IG_BUSINESS_ACCOUNT_ID`**

Part 5: Secret Manager に登録

Step 12. Secret Manager に追加

08_secret_manager.md で未登録の2つを追加:

⑨ `ig-access-token`

- 値: Step 10 で取得した **長期トークン** (`EAAxxxxxxxxxxx...`)

⑩ `ig-business-account-id`

- 値: Step 11 で取得した **ビジネスアカウントID** (`17841xxxxxxxxxxx`)

Step 13. Cloud Run を再デプロイ (環境変数追加)

09_cloud_run_deploy.md Step 5 のコマンドに、以下を追加:

```
gcloud run services update manabi-ops \
  --region asia-northeast1 \
  --update-secrets "IG_ACCESS_TOKEN=ig-access-token:latest,IG_BUSINESS_ACCOUNT_ID=ig-business-account-
id:latest"
```

Step 14. 動作確認

1. Cloud Run URL + `/healthz` にアクセス
2. `"ig": true` になっていればOK
3. LINE で写真を送って **[⊠投稿]** ボタンを押す → 30秒以内に Instagram に投稿される

Step 15. Meta アプリ審査 (本番公開)

開発中は Meta アプリは「開発モード」で、投稿できるのはアプリ管理者のみ。
外部ユーザーが見る Instagram 投稿 として使う場合は **「本番モード」** への切り替えが必要。

審査が必要な権限

- `instagram_content_publish` (投稿機能)
- `pages_read_engagement`
- `business_management`

審査フロー

1. アプリダッシュボード → **「アプリレビュー」** → **「権限と機能」**
2. 各権限の **「リクエスト」** ボタンをクリック
3. 以下を提出:
   - **プライバシーポリシーURL** (要作成)
   - **データ削除手順URL** (要作成)
   - **デモ動画** (機能の動作を撮影・2分以内)
   - **ユースケース説明** (日本語OK)
4. 審査期間: **5営業日〜2週間**
5. 承認されるとアプリを「本番モード」に切り替え可能

プライバシーポリシー・データ削除手順の雛形

→ ManabiOps 運営代表者に別途共有 (リポジトリ `docs/legal/` に格納予定)

☒完了チェック

- [ ] Facebook Page 「まなびキッチン」作成
- [ ] Instagram をビジネスアカウント化
- [ ] Facebook Page と Instagram を紐付け
- [ ] Meta アプリ `ManabiOps` 作成
- [ ] 長期アクセストークン取得 (60日)
- [ ] Business Account ID 取得
- [ ] Secret Manager 登録 (`ig-access-token` , `ig-business-account-id`)
- [ ] Cloud Run 再デプロイ → `/healthz` で `ig: true`
- [ ] テスト投稿成功
- [ ] (本番運用時) Meta アプリ審査完了

☒60日後のトークン更新

長期トークンは **60日間のみ有効**。期限前に更新が必要です。

自動更新オプション (推奨)

ManabiOps では、Cloud Scheduler に月1回の `/internal/refresh-ig-token` タスクを追加予定 (Phase 3 後半)。
手動更新の場合は、Step 10 を繰り返し、Secret Manager のバージョンを更新。

☒次のステップ

→ **13_troubleshooting.md** — 動作確認・トラブルシューティング

🆘 トラブルシューティング

☒ `instagram_content_publish` 権限がリクエストできない

- アプリが「ビジネス」タイプで作成されているか確認
- Instagram が Business アカウント化されているか
- Facebook Page と Instagram が紐付いているか

☒ `Error validating access token: The session has been invalidated`

- トークンの60日が切れている → Step 10 で再取得
- Facebook パスワードを変更した → トークンが無効化される

☒投稿時に `Application does not have permission for this action`

- Meta アプリ審査が必要 (Step 15)
- 開発モードでは、アプリ管理者の Instagram にのみ投稿可能

☒投稿はされるが動画がリールにならない

- 動画の長さが3秒〜90秒か確認 (リール仕様)
- アスペクト比が 9:16 に近いか確認

13. 動作確認・トラブルシューティング

所要時間: 問題発生時の随時参照
前提: 09_cloud_run_deploy.md, 10_line_webhook.md 完了

☒このガイドでやること

ManabiOps が本番稼働した後の **動作確認手順** と、よくある問題の **解決方法** をまとめます。

Part 1: 動作確認チェックリスト

デプロイ直後の導通テストとして以下を順にチェックしてください。

① ヘルスチェック

```
curl https://manabi-ops-xxxxxx-an.a.run.app/healthz
```

期待される出力:

```
{
  "ok": true,
  "mode": "production",
  "services": {
    "line": true,
    "gemini": true,
    "docai": true,
    "firestore": true,
    "sheets": true,
    "gcs": true,
    "ig": false  ← Phase 3 未実施なら false でOK
  }
}
```

- [ ] `ok: true`
- [ ] `mode: production`
- [ ] Phase 1 で必要な全サービス (line/gemini/docai/firestore/sheets/gcs) が `true`

---

② 管理画面ログイン

1. ブラウザで `https://manabi-ops-xxxxxx-an.a.run.app/admin`
2. Basic認証を要求される → Secret Manager で設定した `admin-basic-auth` 値 (`admin:password`) を入力
3. 管理ダッシュボードが表示される

確認項目:
- [ ] ユーザー一覧に初回登録の Owner が表示される
- [ ] 承認待ち一覧が表示される (空でOK)
- [ ] ストレージ使用量が表示される

---

③ LINE 経由のレシート取引E2Eテスト

1. LINE アプリで「まなびキッチン」公式アカウントを開く
2. 任意のレシート画像 (実物写真でもネット画像でも可) を送信
3. **30秒以内** に Flex Message が返ってくる
4. **[☒登録]** ボタンをタップ
5. Google Sheets を開いて、新しい行が追加されているか確認

確認項目:
- [ ] 画像送信 → 返信 (画像認識応答)
- [ ] OCR 結果が Flex Message で表示される
- [ ] 承認後、Sheets に行が追加される
- [ ] Firestore Console の `transactions` コレクションにもレコードがある

---

④ LINE 経由の参加者名簿テスト (Phase 2 以降)

1. LINE に手書き名簿画像を送信
2. Flex Message で参加者リストが返ってくる
3. **[☒登録]** で Firestore `participants` に暗号化保存される

---

⑤ LINE 経由の写真投稿テスト (Phase 3)

1. LINE にイベント写真を送信
2. Flex Message で顔ぼかし済み写真 + 自動生成キャプションが返る
3. **[☒投稿]** で Instagram に30秒以内に投稿

---

⑥ バッチ動作テスト

1. GCP Console → **Cloud Scheduler** → `manabi-ops-hourly-batch`

2. **「今すぐ実行」** → 「成功」になるか

Part 2: よくあるエラーと解決方法

⊠E1. LINE メッセージを送っても返事が来ない

原因候補 (発生頻度順):

1⊠LINE の応答メッセージ設定が干渉

- LINE Official Account Manager にログイン
- **応答設定** → 「応答メッセージ」= **オフ**, 「Webhook」= **オン**

2⊠Webhook URL が誤り

- LINE Developers Console → Messaging API 設定 → Webhook URL
- `https://manabi-ops-xxxxxx-an.a.run.app/webhook` で末尾 `/webhook` が付いているか

3⊠Cloud Run がダウン

```
curl https://manabi-ops-xxxxxx-an.a.run.app/healthz
```

応答がない場合:

```
gcloud run services logs read manabi-ops --region asia-northeast1 --limit 50
```

でエラーログを確認。

4⊠Webhook 署名検証エラー

- Secret Manager の `line-channel-secret` 値が正しいか
- 前後の空白・改行を含めていないか
- 修正後: `gcloud run services update manabi-ops --region asia-northeast1` で再反映

⊠E2. 「権限がありません」エラー (EX-10)

- ホワイトリストに LINE User ID が未登録
- 解決手順:
  1. LINE 管理者が `/adduser 名前 U_xxxx` コマンドを送信
  2. または管理画面 `/admin/users` から追加

⌧E3. OCR 結果が不正確

レシート

- Document AI の Expense Parser は **日本語レシート対応** だが精度は画像品質に依存
- **明るい場所で・まっすぐ・平らに** 撮影するよう運用ルール化
- 金額が誤検出される場合 → 管理画面 `/admin/transactions` から手動修正

名簿 (Custom Extractor)

- 訓練データが不足している可能性
- 追加で10〜20枚サンプルを与えて再訓練 (04_document_ai.md Part 2)

⌧E4. Google Sheets にデータが書き込まれない

1. Sheets へのサービスアカウント共有が **編集者** か確認
2. `SHEETS_SPREADSHEET_ID` が正しいか確認
3. シート名が `ledger` (半角英字) か確認
4. Cloud Run ログで `sheets:` 関連エラーを検索

```
gcloud run services logs read manabi-ops --region asia-northeast1 --limit 100 | grep -i sheets
```

⌧E5. 顔ぼかしが適用されない / 処理が遅い

- MediaPipe が顔を検出できない場合 (斜めの顔、マスク等) → OpenCV fallback が動くが精度低下
- Cloud Run メモリ不足 → `--memory 2Gi` に増量
- コールドスタート遅延 → `--min-instances 1` に設定 (月¥1,500程度加算)

⌧E6. Instagram 投稿失敗

- Meta トークンの60日期限切れ → 12_instagram_graph_api.md Step 10 で再発行
- アプリがまだ開発モード → 12_instagram_graph_api.md Step 15 (審査)
- 動画の尺・サイズがリール仕様を満たしていない

⌧E7. 月額コストが想定より高い

1. GCP Console 課金 で項目別確認
2. 主因の候補:
   - **Document AI**: 月1,000ページ超過 → レシート枚数を確認

- **Cloud Storage**: 古いファイルが溜まっている → ライフサイクル設定確認 (06)
- **Cloud Run**: `min-instances` を1以上にしている → 0に戻す

3. 月額上限アラート設定: GCP 課金 → **「予算とアラート」** → ¥10,000 で通知

Part 3: ログの見方

Cloud Run ログ

```
# 直近50件
gcloud run services logs read manabi-ops --region asia-northeast1 --limit 50

# リアルタイム監視（別ターミナルで実行）
gcloud run services logs tail manabi-ops --region asia-northeast1

# 特定キーワード検索
gcloud run services logs read manabi-ops --region asia-northeast1 --limit 500 | grep -i error
```

GCP Console でログ検索

1. GCP Console → **Cloud Logging** → **ログエクスプローラ**
2. 左上リソースで `Cloud Run Revision` → `manabi-ops` を選択
3. 「クエリ」欄で `severity=ERROR` 等を指定

ログのフィルタ例

```
# エラーだけ
severity=ERROR

# OCR関連
resource.labels.service_name="manabi-ops" AND jsonPayload.component="docai"

# 特定ユーザーの動作
jsonPayload.userId="U_xxxxxxxx"
```

Part 4: 復旧手順

🆘 Cloud Run サービスが完全に落ちている

1. Cloud Run Console を開く
2. `manabi-ops` サービスを選択
3. **「リビジョン」** タブで、直近の成功リビジョンに戻す:
   - 動いていたリビジョン行の ⋮ → **「トラフィックを100%」**
4. または再デプロイ:

```
gcloud run deploy manabi-ops --source . --region asia-northeast1
```

🆘 Secret Manager のキーを誤って上書きした

1. Secret Manager → 該当シークレット → **「バージョン」** タブ
2. 前のバージョン (数字が1つ小さい) の行 → **「...」** → **「このバージョンにロールバック」**
3. Cloud Run を再起動:

```
gcloud run services update manabi-ops --region asia-northeast1
```

🆘 Firestore に間違ったデータを入れた

- **1件だけ**: Firestore Console でドキュメントを開いて削除/編集
- **一括削除**: 管理画面 `/admin/firestore` (データ削除タブ) から削除 — Owner 権限のみ
- **全コレクション**: GCP Console → Firestore → コレクションを選択 → **データを削除**

Part 5: 運用チェックリスト (月次)

毎月1日に以下を確認 (月次サマリーと一緒に):

- [ ] GCP 月額コストが ¥10,000 以下
- [ ] Cloud Storage 使用量が 5GB 以下 (無料枠内)
- [ ] Firestore 書込数が2万/日を超えていない
- [ ] Document AI 使用量が1,000ページ/月を超えていない
- [ ] Instagram トークン有効期限 (残り30日切ったら更新準備)
- [ ] Cloud Scheduler ジョブ3つすべて「成功」継続
- [ ] 月次サマリー LINE が届いている (FR-11)
- [ ] エラーログの傾向確認 (異常増加していないか)

☒次のステップ

セットアップ完了 ☒

- 継続運用: 本ガイドをブックマーク
- 新機能追加: GitHub PR で開発者と相談
- データ分析: 11_looker_studio.md (任意)

🆘 それでも解決しない場合

以下を開発者に連絡:

1. **発生時刻** (JST)
2. **何をしたか** (操作手順)
3. **期待した結果**
4. **実際に起きたこと**
5. **エラーメッセージ全文**
6. **スクリーンショット**
7. Cloud Run ログ (直近100件を `gcloud logs read` で取得)

14. 運用開始チェックリスト

> **対象**: まなびキッチン 運営代表者・主担当者
> **利用時**: 本番稼働開始前、および月次運用確認時

☒このガイドでやること

ManabiOps を **本番運用として開始する前の最終確認** と、
継続運用時の **月次/年次チェックリスト** をまとめます。

Part 1: 本番稼働前 最終確認 (Go/No-Go)

以下 **すべて** ☒になれば本番稼働OK。1つでも ☒があれば稼働保留。

A. インフラ

- [ ] GCP プロジェクト `manabi-ops` が作成済み
- [ ] 請求アカウントがリンク済み、予算アラート ¥10,000 設定済み
- [ ] 必要な API が全て有効化済み (Firestore, Cloud Run, Document AI, Sheets, Storage, Secret Manager, AI Platform)
- [ ] サービスアカウント `manabi-ops-sa` 作成済み
- [ ] Cloud Run サービス `manabi-ops` が稼働中 (asia-northeast1)
- [ ] Cloud Run が `--min-instances 0` で設定されている (無料稼働)
- [ ] `/healthz` が `{"ok":true,"mode":"production"}` を返す
- [ ] 必要なサービスが全て `true` (line/gemini/docai/firestore/sheets/gcs)

B. シークレット

- [ ] Secret Manager にPhase 1必須の6シークレット登録済み
- [ ] `encryption-key` を **オフラインで2箇所以上** バックアップ済み (USB + 紙)
- [ ] `admin-basic-auth` パスワードが16文字以上
- [ ] サービスアカウントに `Secret Accessor` ロール付与済み

C. LINE

- [ ] LINE公式アカウント「まなびキッチン」作成済み
- [ ] Webhook URL 設定 + 検証成功
- [ ] Webhook 利用 = オン, 応答メッセージ = オフ
- [ ] 運営代表者が初回メッセージで Owner 登録済み
- [ ] テスト用レシート画像で E2E 成功

D. Google Workspace

- [ ] Google Sheets `まなびキッチン 会計台帳` 作成済み
- [ ] シート名 `ledger`, ヘッダー10列入力済み
- [ ] サービスアカウントに編集者権限共有済み
- [ ] `SHEETS_SPREADSHEET_ID` が Secret Manager に登録済み

E. Document AI

- [ ] レシートプロセッサ (Expense Parser) 作成済み
- [ ] Processor ID が Secret Manager に登録済み
- [ ] 実レシートで90%以上の精度確認済み (受入基準 AC-01)

F. Cloud Scheduler

- [ ] 3ジョブ (hourly / monthly / daily) 作成済み
- [ ] すべて「今すぐ実行」テスト成功
- [ ] タイムゾーン `Asia/Tokyo` 設定

G. 管理画面

- [ ] `/admin` に Basic認証でアクセス可能
- [ ] ユーザー管理画面で Owner 以外のスタッフを1〜2名追加済み
- [ ] 承認待ち一覧が表示される

H. 運用ルール

- [ ] 運営代表者以外の承認権限者が2名以上 (Bus Factor対策)
- [ ] 承認者の LINE User ID が管理画面に登録済み
- [ ] トラブル時の連絡先が明文化されている
- [ ] 月次確認担当者が決まっている

I. ドキュメント

- [ ] README.md 最新

- [ ] セットアップガイド (本ディレクトリ) 最新
- [ ] `.env.example` 最新 (機密情報は含めない)
- [ ] 個人情報保護ポリシーを運営内で周知

Part 2: 月次運用チェックリスト (毎月1日)

#	項目	確認方法	目安
1	GCP 月額コスト	GCP 課金	¥3,000 以下推奨
2	Cloud Storage 使用量	Storage Console	5GB 以下
3	Firestore 書込数	Firestore Usage	1万/日以下
4	Document AI 使用ページ数	Document AI Console	月1,000以下
5	Cloud Scheduler ジョブ正常稼働	Scheduler 画面	3ジョブ全て成功
6	エラーログ数	Cloud Logging (severity=ERROR)	先月比で急増していない
7	月次サマリーLINE受信	LINE	毎月1日 9:00 に届く
8	Instagram 投稿件数・エンゲージ	Instagram Insights	—
9	参加者累計・新規数	管理画面 /admin/stats	—
10	LINEトーク返信成功率	ログ検索	95% 以上

Part 3: 半年ごと (4月・10月)

- [ ] GCP サービスアカウントキー ローテーション
- [ ] `admin-basic-auth` パスワード変更
- [ ] 運営メンバー変更を管理画面に反映
- [ ] Firestore `participants` の不要データ棚卸し
- [ ] Cloud Storage 内の肥大化フォルダ確認

Part 4: 60日ごと (Phase 3 運用時)

- [ ] Instagram アクセストークン更新 (12_instagram_graph_api.md Step 10)
- [ ] Meta アプリの審査状況確認

Part 5: 年次 (3月末 or 年度末)

- [ ] 年次集計エクスポート (会計台帳CSV)
- [ ] Firestore 全データバックアップ (GCP Firestore Export)
- [ ] Cloud Storage → AI Drive 長期保管 (必要分のみ)
- [ ] 要件定義書 v1.0 → v1.x 改訂検討
- [ ] 依存ライブラリ更新 (npm audit, `npm update`)
- [ ] Node.js LTS 最新版へのアップグレード検討

Part 6: 緊急時対応 (インシデント発生時)

Step 1. 事実確認

- いつから? (開始時刻)
- 何が? (現象)
- 範囲は? (全ユーザー / 一部 / 特定機能のみ)

Step 2. 一次対応

状況	対応
Cloud Run ダウン	前リビジョンにロールバック (13_troubleshooting.md Part 4)
LINE 返信停止	Webhook URL + 応答設定を再確認
データ誤登録	管理画面から削除 + 再入力
個人情報漏洩疑い	**直ちに Cloud Run を停止**、開発者連絡
Instagram 誤投稿	Instagram アプリから削除

Step 3. 復旧後

- インシデントレポート作成 (発生→原因→対応→再発防止)
- 必要に応じて参加者・保護者への連絡
- `CHANGELOG.md` に記録

Part 7: Bus Factor 対策

1人の担当者しか運用できない状態を避ける ため:

- [ ] **最低3名** が管理画面ログイン可能

- [ ] **最低2名** が Cloud Run デプロイ操作可能 (gcloud CLI インストール済み)
- [ ] パスワード類は **1Password / Bitwarden** 等パスワードマネージャで共有
- [ ] `encryption-key` のオフラインバックアップ場所を複数人が把握
- [ ] 本ドキュメントを全員が読んでいる (読了チェック)

Part 8: 退任・引継ぎ時

担当者の交代があった場合:

- [ ] GCP IAM から退任者のアカウント削除
- [ ] LINE ホワイトリストから退任者の User ID 削除
- [ ] 管理画面パスワード変更
- [ ] サービスアカウントキー再発行 (疑いがあれば)
- [ ] Google Sheets 共有から退任者削除
- [ ] Meta アプリ (Phase 3) の管理者から退任者削除

☒完了したら...

セットアップ完了 ☒
本ドキュメントをブックマーク・印刷して、月1回レビューしてください。

☒その他のガイド

- 00_overview.md — 全体概要
- 13_troubleshooting.md — トラブル対応
- README.md — ガイド一覧に戻る

このドキュメントは ManabiOps プロジェクトの成果物です。
© 2026 まなびキッチン / MIT License